# Using Deep Neural Networks to Predict Pressure Coefficients of Low Reynolds Number Airfoils

**BEng (Hons) Mechanical Engineering**

**Faculty of Computing, Engineering and the Built Environment**

**School of Engineering and Built Environment**


**Student Name:** Jay Parekh

**Student Number:** S14140141


**Submission Date:** 06/2020

**Supervisor:** Dominic Flynn

Dissertation submitted to Birmingham City University in partial fulfilment of the requirements for the degree of Bachelor of Engineering

**Abstract**

A DNN model with 2 hidden layers (256 to 128 Neuron) was designed to quickly and accurately predict the pressure coefficients of 2D airfoils at an angle of attack of 6.5°, with a flow speed of Mach 0.1 and a Reynolds number of 50000. Diffeent configurations of deep neural; networks were compared to find the optimal configuration. The chosen DNN model was found to be 97.58% accurate, which showed a 50% improvement over the error margin CNN model designed by Zelong et al (2018). The model used exponential linear unit activation rather than the rectified linear unit activation used by Zelong et at. (2018) although the model was implicated by a high count of errors when the pressure coefficient was at a value which was less than minus one.

**Acknowledgements (Note: only for final year reports, not assignments)**

I'd like to thank my tutors and family for supporting me.

**1 Glossary ..... v**

**2 Introduction ..... 1**

2.1 Background and problem definition (REF UP) ..... 1

2.2 Scope of the project ..... 1

2.3 Rationale ..... 2

2.4 Aim and objectives ..... 2

**3 Review of Existing Knowledge ..... 3**

**4 Methodology ..... 13**

**5 Design Development ..... 14**

**6 Results and Analysis ..... 28**

**7 Discussion ..... 30**

**8 Conclusions ..... 31**

**9 Recommendations ..... 33**

**10 References ..... 34**

**11 Appendix ..... 37**

# 1 Glossary

| | |
|---|---|
| ***Deep neural network*** | *A neural network with more than one layer.* |
| ***Feature set*** | The data used as identifiers, in order to establish trends for the predicted data. |
| ***Label set*** | The data to be predicted, by the deep neural network. |
| ***Data Frame*** | The location at which the data is stored for use by the deep neural network. |
| ***Count plot*** | A graph that shows value counts of instances. |
| ***Correlation matrix*** | A diagram that shows an easy to interpret correlation for the feature and label sets. |
| ***Hidden layer*** | The layer(s) between the input and output layer of a neural network. |

# 2 Introduction

## 2.1 Background and problem definition.

Low Reynolds number airfoils have become increasingly popular, for hobby use, reconnaissance (Winslow et al., 2018), as well as for transport on Mars (Désert et al., 2017), as the viability of colonizing mars becomes more realistic (Levchenko et al., 2018). The atmospheric surface density of Mars is 0.02kg/m$^3$ (NASA, 2020), while Earth's is 1.217kg/m$^3$ (NASA, 2020), over six times denser. The result of this is that the Reynolds number of the flow around airfoils on Mars, in comparison to Earth, is very low, forcing aerodynamicists to consider ultra-low Reynolds number airfoils (Winslow et al., 2018). For example, a micro air vehicle (MAV) on Earth uses a Reynolds number of 50000 for simulations (Désert et al., 2017), while on Mars, the Reynolds number used is 3000 (Désert et al., 2017). Due to the low Reynolds number, more thrust is required from the engine(s) to maintain flight, while the Earth equivalent engine(s) on Mars, will produce less power due to less air available for combustion (Désert et al., 2017), stressing the importance of using high lift, low Reynolds number airfoils. To analyze an airfoil's suitability, aerodynamicists inspect their pressure distribution. Usually obtained through a simulation on XFoil, the process of smoothing and parametrization is arduous, reducing time available for research. Using deep neural networks, a regression model is to be produced, which can predict the pressure coefficients of 2D airfoils, given their co-ordinate data, provided the parameters are constant.

## 2.2 Scope of the project.

The project covered the smoothing and parametrization of airfoil co-ordinate data in order to obtain XFoil simulations of the pressure coefficient data, after which the data was graphically represented to visualize the distribution and correlation of the data. The obtained data was converted to be used in a deep neural network and used as an input to several configurations of deep neural networks to find the optimal configuration via the evaluation of the loss plots, root squared scores and prediction plots, of the models.. The optimal model was then planned to be deployed onto the Microsoft Azure (Microsoft, 2020) cloud server to allow for scalability of the model, allowing the remainder of the airfoil data to be added to the data frame and for the model to be computed effectively. Aerodynamicists would benefit from the convenience of the

regression model as the effect of changing geometric factors of the 2D airfoils can be evaluated with much less time spent carrying out simulation work, increasing productivity and the rate of new discoveries due to more time spent evaluating results. The project was planned to take a duration of 287 working days, which was beyond the timeframe set for the completion of the project. The focus of the project was to design a deep neural network which uses regression rather than classification to predict the pressure coefficients of airfoils with greater accuracy than the classification method.

## 2.3 Rationale.

Obtaining pressure coefficients from XFoil is time consuming due to smoothing and parametrization. Artificial intelligence and machine learning methods, will be adapted to the problem to reduce the time taken for airfoil analysis by running predictions, saving aerodynamicists precious time. Although image recognition models were previously researched (Bhatnagar et al., 2019, there was a lack research on regression based DNN models, which use continuous values, rather than images.

## 2.4 Aim and objectives.

**Aim.**

To design and deploy a regression DNN model, to predict the pressure distribution of low Reynolds number 2D airfoils.

**Objectives.**

- Obtain and classify by optimal angle of attack, 100 low Re airfoil co-ordinate files from Airfoil Tools.
- Use XFoil to smooth the airfoils using the design parameters and obtain their pressure coefficients.
- Convert the XFoil data to a Pandas data frame format, inspecting and visualizing the data frames.
- Visualize the distribution and correlation heatmap of the data frame
- Design, compile, train, test and validate models, recording loss, predictions and error.

- Evaluate the performance of each model, using Mean Squared Error, selecting a final model.

# 3 Review of Existing Knowledge.

A literature review was undertaken to assess the viability of each source as well as the action taken as a result of the research. The literature review evaluates the sources used for setting the parameters for simulations on XFoil, as well as considerations made for the design process of the deep neural network model.

**Hands-On Machine Learning with Scikit-Learn, Keras & Tensorflow** (Géron, 2019).

**Defining the literature used.**

Géron (2019) emphasised the importance of setting assumptions and appropriate performance measures at the beginning of machine learning projects, as well as understanding data types. The textbook featured an “end-to-end machine learning project” (Géron, 2019), which explained the stages within the process, as well as information on configuring artificial neural networks, and the architectures available. Géron (2019) also discussed performance optimizations such as gradient boosting and “hyperparameter tuning” (Géron, 2019).

**Why the literature was used.**

The textbook was used in order to build the skills required for producing a functional artificial neural network with appropriate performance measures for real world applications. The textbook was also used to understand which artificial neural network solutions were appropriate for the continuous data collected. Gradient boosting techniques were found to improve the training time and accuracy of machine learning models and were compatible with the feature set. Hyperparameter tuning using GridSearchCV improved the accuracy of models by adjusting the parameters of the model automatically and cross validating the model to find the optimal conditions.

**How the literature was used.**

The textbook was used as guidance for the process of producing the deep neural network from start to finish, to understand the concepts involved at each stage. The textbook drove the decision of using models that were compatible with continuous variables, which were regression models. The adamax optimizer was chosen to be

used for the deep neural network model to be designed. Hyperparameter tuning was to be implemented to the completed model to improve prediction accuracy.

**Aerodynamic Coefficient Prediction of Airfoils with Convolutional Neural Network, (Zelong et al., 2018).**

**Defining the literature used.**

Zelong et al. (2018) had investigated the convolutional neural network architecture for the use of predicting the pressure coefficients of 2D airfoils. Having used “artificial images” as inputs to the algorithm, Zelong et al. (2018) used the rectified linear unit activation function for their convolutional neural network. The convolutional neural network was found to be the most effective configuration at the time of writing, for predicting the pressure coefficient of 2D airfoils. The convolutional neural network designed by Zelong et al. (2018) was found to be 95% accurate with 30000 epochs of training.

**Why the literature was used.**

The literature was used to find guideline parameters to use for the deep neural network that was designed. The activation function was used for the initial consideration of the type of activation functions to be compared. The accuracy score of the configuration was used as a guideline of the score that was aimed for when developing the alternative deep neural network. Finally the designed deep neural network had to use less epochs than the model by Zelong el al. (2018), to be time effective.

**How the literature was used.**

As a linear unit was used as part of the activation of the convolutional neural network, alternative linear unit configurations were considered. The activation functions used were leaky rectified linear units and exponential linear units, as suggested improvements to the rectified linear unit activation function. The deep neural network designed as part of the project was designed to score a greater accuracy than the 95% achieved by Zelong et al. (2018). The models tested used only 1000 epochs in order to reduce testing time, when compared to the convolutional neural network proposed by Zelong et al. (2018).

**Prediction of Aerodynamic Flow Fields Using Convolutional Neural Networks** (Bhatnagar et al., 2019).

**Defining the literature used.**

The format of data, artificial images which accounted for the flow speed, angle of attack and Reynolds number of each airfoil, was taken note of. During the project, a “convolutional neural network” (Bhatnagar et al., 2019) was trained for “30000 epochs” (Bhatnagar et al., 2019) to predict the flow fields at random points along the airfoil. The model found that the flow fields were “4 orders of magnitude” (Bhatnagar et al., 2019) faster than the original “Reynolds Averaged Navier Stokes solver” (Bhatnagar et al., 2019), with a “10% average” (Bhatnagar et al., 2019), mean squared error.

**Why the literature was used.**

The report signifies the analysis time reduction compared to the “Navier Stokes” (Bhatnagar et al., 2019) machine learning model. A classification model was used, in the form of a convolutional neural network which was one of the options considered during background research. It also discusses the parameters used for training the neural network model, as well as its accuracy. It was also found that the performance of artificial neural networks is better than using traditional machine learning methods.

**How the literature was used.**

The article was used as a guideline to the parameters to be set, as well as a margin of error to improve upon. The convolutional neural network architecture was not viable as the process of artificial image generation was complex. The journal article drove the decision to design a deep neural network model for the project

**Basic Understanding of Airfoil Characteristics at Low Reynolds Numbers (104–105)** (Winslow et al., 2018).

**Defining the literature used.**

The article describes the increase in popularity for the use case of micro-aerial vehicles, such as "compact electronics" (Winslow et al., 2018) and "improved power storage" (Winslow et al., 2018). It also briefly reviews “military use” (Winslow et al., 2018) cases such as “reconnaissance” (Winslow et al., 2018). The article also features a diagram which shows the effect of the transition of Reynolds number from laminar to turbulent flow. Winslow et al. (2018), also describes how low Reynolds number operating conditions “incur higher power requirements”(Winslow et al., 2018) when using “conventional airfoils” (Winslow et al., 2018) due to an excess of “aerodynamic drag”. In the conclusion Winslow et al. (2018) stresses the importance of

understanding how airfoil performance is affected by “geometric variations, such as camber and thickness" (Winslow et al., 2018).

**Why the literature was used.**

The journal article was used to identify use cases of low Reynolds number airfoils such as micro-aerial vehicles, used for reconnaissance. The article stresses the need to understand the performance of low Reynolds number airfoils, as well as the contributing factors to changes in performance.

**How the literature was used.**

The journal article was used as part of the background to the report, in order to identify the practical applications of low Reynolds number airfoils. The article was also used to understand the effect of changing the of Reynolds number and the requirement for high lift airfoils. Finally the article was leveraged when selecting airfoils for the dataset, by selecting sets of airfoils with varying camber and thickness.

**Mars Colonization: Beyond Getting There** (Levchenko et al., 2018).

**Defining the literature used.**

The article reviews the legal, ethical and physical challenges faced for the colonization of Mars. Levchenko et al. (2018) describe the colonization of Mars as a “real necessity” (Levchenko et al., 2018), stating how it was essential to focus on “using space technology to positively contribute to our quality of life” (Levchenko et al., 2018) when living on Mars.

**Why the literature was used.**

The article was used to understand the challenges faced with the colonization of Mars, in order to evaluate the appropriateness of the use cases presented in the project background.

**How the literature was used.**

The article enforced the fact the colonization of Mars was close, but there were many challenges to be faced. Improving research methods in the field of low Reynolds number airfoils would contribute to solving the engineering challenges faced when designing Martian aircraft, which made the use case appropriate.

**Aerodynamic Design of a Martian Micro Air Vehicle** (Désert et al., 2017).

**Defining the literature used.**

The article describes the flow characteristics required for a Martian air vehicle (Désert et al., 2017), in comparison to an air vehicle on Earth. The article discusses the use of

ultra-low Reynolds number airfoils (Désert et al., 2017) for the low atmospheric surface density on Mars.

**hy the literature was used.**

The article was used in the process of considering the appropriate Reynolds number to use when running XFoil simulations for the project.

**How the literature was used.**

The article provided further justification to use a Reynolds number of 50000, as an appropriate use case was presented.

**XFOIL vs CFD performance predictions for high lift low Reynolds number airfoils** (Morgado et al., 2016).

**Defining the literature used.**

The article compared the accuracy of 50, 100, 150 and 200 point smoothed airfoil performance simulations using XFoil (Morgado et al., 2016).

**Why the literature was used.**

The journal article was used to understand the effect of increasing the number of panel-nodes on the accuracy of simulations.

**How the literature was used.**

It was found that using more panel-nodes improved accuracy, which drove the decision of using the maximum possible number of panel-nodes for the simulations during the project, as input data.

**Reynolds Number Effects on the Performance of Small-Scale Propellers** (Deters et al., 2014).

**Defining the literature used.**

The article began by explaining the use case of low Reynolds number airfoils, which was found to be unmanned aerial vehicles and manned aerial vehicles (Deters et al., 2014). The article evaluates the effect of changing Reynolds numbers on small propellers. The article demonstrated that as the Reynolds number increased, the pressure coefficient decreased, due to decreased fluid viscosity (Deters et al., 2014).

**Why the literature was used.**

The article provided use cases for low Reynolds number airfoils and provided understanding for the effect of changing the Reynolds number, on the pressure distribution.

**How the literature was used.**

From the article it was learnt that the Reynolds number had to remain consistent throughout the data collection process, to ensure the predictions were accurate.

**Design Approach for Selection of Wing Airfoil with Regard to Micro-UAVs (**Brusov et al., 2011).

**Defining the literature used.**

When selecting the data, Brusov et al. (2011) had selected low, medium and high-speed airfoils to adopt a "multitask optimization" (Brusov et al., 2011) strategy. The airfoils used were all low Reynolds number airfoils, featuring a high lift to drag ratio. Optimizing the selection of the airfoil used for the micro-unmanned aerial vehicle showed improvements in efficiency as well as airspeed, due to reduced drag (Brusov et al., 2011).

**Why the literature was used.**

Brusov et al. (2011) used the Reynolds number, angle of attack and flow speed as variables, which increased the model's complexity. The article also provided a use case for low Reynolds number airfoils.

**How the literature was used.**

The article drove the decision of setting the speed parameter of XFoil to a low speed as well as the decision to keep parameters constant to reduce model complexity. The use case of the micro-unmanned aerial vehicle was used as part of the background of the report.

**Artificial Neural Network Investigation of Aerodynamic Pressure Load Modelling** (Chen, 2009).

**Defining the literature used.**

When selecting the data, similar airfoils were used for the artificial neural network to improve its accuracy. The data was prepared by scaling the values using MinMaxScaler (Chen, 2009), including all flight parameters. The artificial neural network featured 2 hidden layers with 10 neurons (Chen, 2009) accounting for the features used. Chen (2009), found that a lack of data had reduced the accuracy of their model and found the process of obtaining the data using XFoil to be laborious (Chen, 2009).

**Why the literature was used.**

The article provided an example of data scaling using minmax scaler, using high resolution co-ordinate data, as well as using similar airfoils to improve the accuracy of

predictions. The artificial neural network used in the model used similar input features, namely the co-ordinates of the airfoil and their pressure coefficients.

**How the literature was used.**

It was learnt that the scaling technique delivered good results, thus MinMaxScaler was used to scale the feature set of the collected data. It was planned that a deep neural network with 2 layers would be tested as part of the model evaluation process.

**Pandas, (2021) .**

**Defining the literature used.**

Pandas is a “data analysis and manipulation tool” (Pandas, 2021). Pandas was required to convert dataframes, such as when the XFoil file was converted to pickle format, as well as visualize the statistics of the data, such as the mean values and row counts. Pandas was also used for appending the co-ordinate data into a single data frame.

**Why the literature was used.**

The Pandas website provided examples of code, which were used to improve the efficiency of the data manipulation process hence reducing the time spent on repetitive tasks.

**How the literature was used.**

By using the commands that were provided on the Pandas website, vast amounts of time were saved with the ability to modify single aspects of large datasets efficiently, such as removing the header row from all co-ordinate files.

**Keras**, (2021)

**Defining the literature used.**

Keras is a “Python deep learning API” (Keras, 2021), which provides complete models as well as all associated parametrization techniques in a single location, along with several examples of their code in practice.

**Why the literature was used.**

The Keras API was used to compile the deep neural network models, which also came from the Keras API. The effectiveness of the activation functions as well as the loss functions and optimizers were evaluated using the context provided by the Keras website.

**How the literature was used.**

The Keras API was used to obtain the dense layers required to produce the deep neural network model as well as both the leaky rectified linear unit and exponential linear unit activation functions. The API also stored the mean squared error loss function as well as the adamax optimizer, which were all used as part of the design conception of the chosen deep neural network.

**Airfoiltools**, (2021)

**Defining the literature used.**

Airfoiltools was a database of airfoil co ordinate data. Obtained from various universities around the world, airfoiltools also provided the facilities to design custom airfoils. The website contained important information regarding the parametrization of airfoil simulations, and provided the optimal angles of attack for a variety of Reynolds numbers, for each airfoil.

**Why the literature was used.**

The literature was used to research the optimal airfoils that were used when airfoil selection was undertaken, while the website also provided the parameters required for the flow conditions of an average wind tunnel.

**How the literature was used.**

An nCrit of 9 was used to imitate the flow conditions of an average wind tunnel, while 100 low speed airfoils were gathered as shown in appendix A, using the optimal angle of attack data as a means of organisation.

**Feature optimization of contact map predictions based on inter-residue distances and U-Net++ architecture** (Shenoy, 2019).

**Defining the literature used.**

A diagram of the comparison of activation functions was used.

**Why the literature was used.**

The diagram was used to observe how the effect of changing the activation function upon the way negative inputs were processed within a deep neural network.

**How the literature was used.**

The literature was used to decide on using an exponential linear unit as the optimal activation function for the deep neural network.

**iDeepLe: deep learning in a flash** (Tahmassebi, 2018).

**Defining the literature used.**

A diagram of the comparison of the accuracy of different optimizers during training and testing, was used.

**Why the literature was used.**

The diagram was used to select the optimal optimizer for the deep neural networks that were configured.

**How the literature was used.**

The literature has shown the adamax optimizer delivered optimal accuracy, thus the adamax optimizer was used.

**The Waterfall Model and the Agile Methodologies : A comparison by project characteristics – short** (Casteren, 2017).

**Defining the literature used.**

Agile and waterfall project management methodologies were compared.

**Why the literature was used.**

The literature was used to consider which project management methodology to use.

**How the literature was used.**

The literature was used to make recommendations at the end of the report, for supporting the use of the agile project management methodology for more effective prototyping.

**Conclusion of literature review.**

In summary, the use cases for low Reynolds number airfoils were: micro-aerial vehicles (Désert et al., 2017) and micro-unmanned aerial vehicles (Brusov et al., 2011) for Earth and for Mars. The chosen Reynolds number of 50000 was also found to be suitable for full size aircraft on Mars, due to its low atmospheric surface density, when compared to Earth (Désert et al., 2017). The chosen airfoils were specified to be closely related in camber and thickness to understand the effect of varying geometry for their pressure distribution (Winslow et al., 2018). Research of low speed, low Reynolds number airfoils are limited (Brusov et al., 2011) , stressing the importance of parameterizing the simulations at low speed, using the highest possible number of panel-nodes for optimal model performance (Morgado et al., 2016). It was decided that the model would use used 2 feature sets, the x and y co-ordinates and the label set of the pressure coefficients. The feature set would be scaled using MinMaxScaler (Chen, 2009) to provide consistency, while the chosen architecture for the project was a deep neural network, the algorithm of which would be gradient boosted using adamax

(Géron, 2019). The model would finally be tuned using GridSearchCV and deployed to a cloud server (Géron, 2019). The sources were all academic and from respectable institutions, which made the data valid.

# 4 Methodology.

Popular within the software engineering industry (Casteren, 2017), the waterfall project management methodology was adopted to plan the project. Each stage of the project was broken down into individual tasks and milestones were set for the achievement of each development stage. The task list was organised in the form of a Gannt chart, which used parent/child relationships throughout to signify future development stages could not be completed without completing the current development stage. Figure one shows the Gannt chart used to plan the project, following the waterfall project methodology.

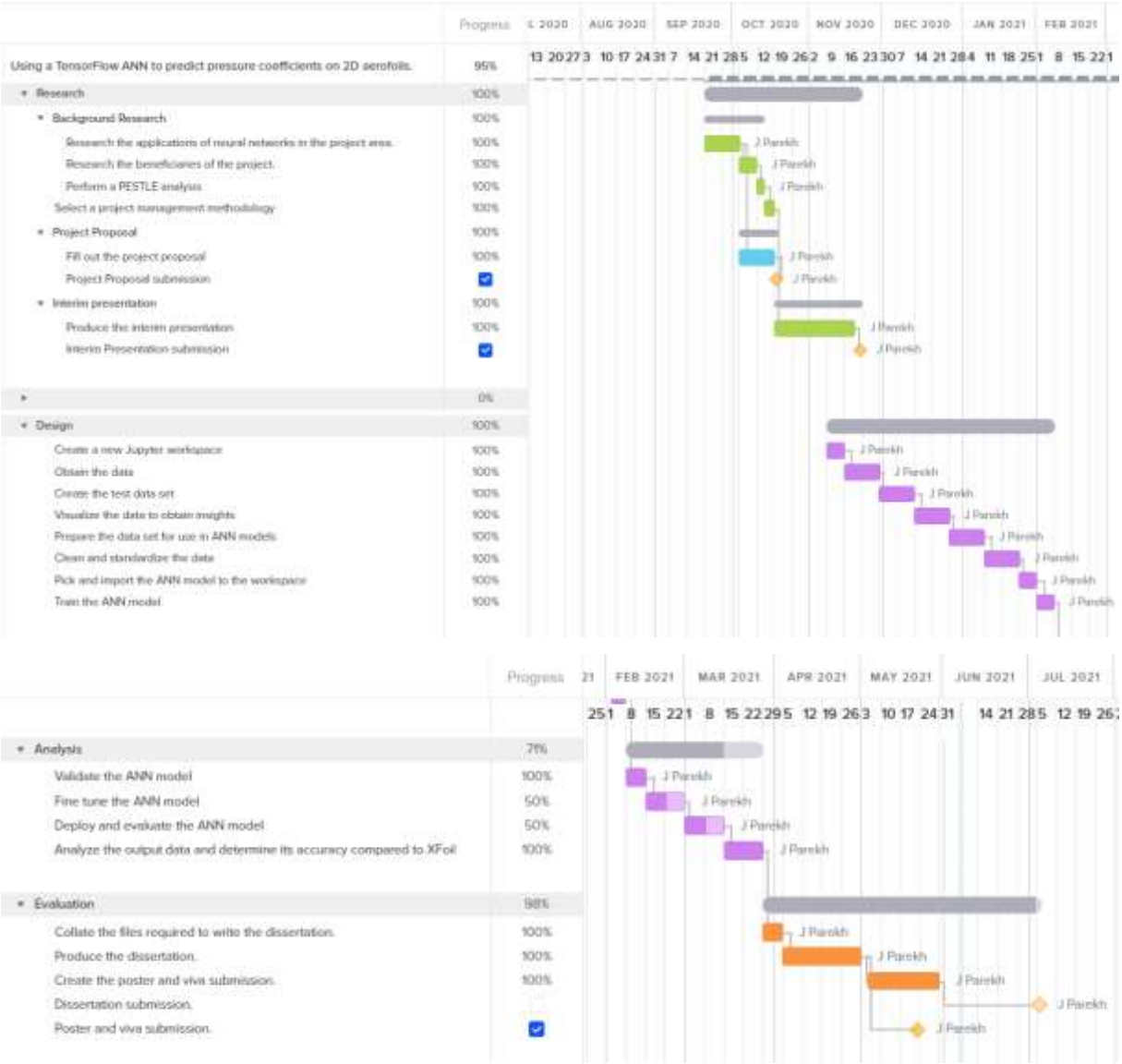



**Figure 1: Gannt chart of the planned project.**

# 5 Design Development.

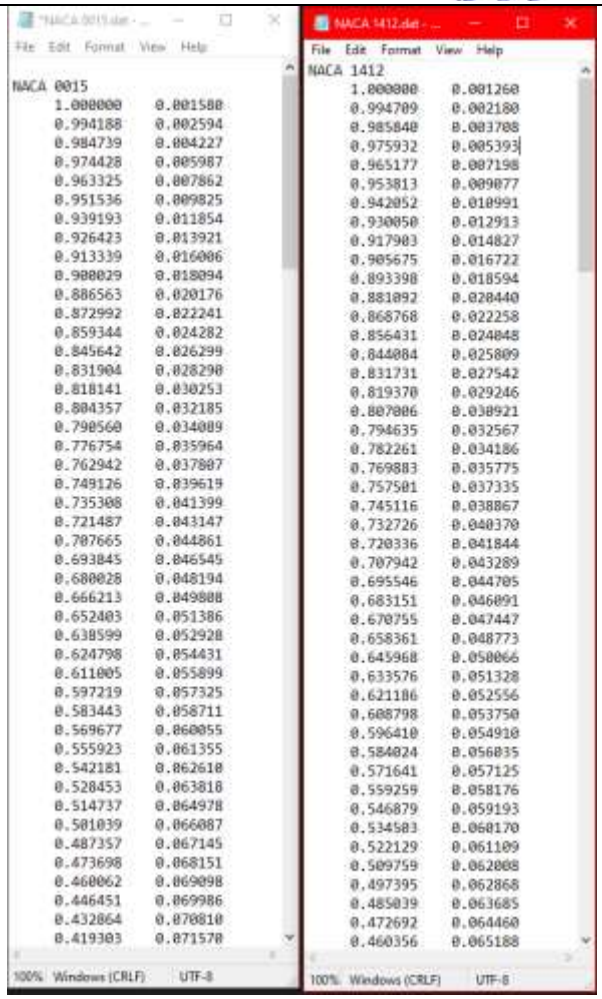
A sample of high lift, low Reynolds number airfoils were evaluated (airfoiltools, 2021), in order to identify airfoils with a common optimum angle of attack, to be used in XFoil for simulation.

The modal angle of attack from the from the sample of data obtained was where α, the angle of attack, in degrees was equal to 6.25. 10 airfoils were selected, which were listed in table 1. The 100 airfoils that were organized by optimal angle of attack, are shown in appendix A.

| Airfoil where α = 6.25 |
|---|
| NACA 0015 |
| Eppler E171 |
| Selig S8038 |
| NACA 1412 |
| Eppler E226 |
| Mike Fox WASP |
| David Fraser DF101 |
| Selig S3016 |
| Selig/Donovan SD7080 |
| Selig/Donovan SD7043 |

**Table 1: List of selected Airfoils.**

Initially it was decided that the number of panel nodes and their panel bunching parameter would be equal. The method would make the x co-ordinates a constant feature, reducing model complexity. Soon it was realised that x co-ordinates vary even with the same panel bunching parameter among the airfoils, as points of greater curvature vary for each airfoil. The observation is represented in figure 2.



```
NACA 0015
    1.000000    0.001580
    0.994188    0.002594
    0.984739    0.004227
    0.974428    0.005987
    0.963325    0.007862
    0.951536    0.009825
    0.939193    0.011854
    0.926423    0.013921
    0.913339    0.016006
    0.900029    0.018094
    0.886563    0.020176
    0.872992    0.022241
    0.859344    0.024282
    0.845642    0.026299
    0.831904    0.028290
    0.818141    0.030253
    0.804357    0.032185
    0.790560    0.034089
    0.776754    0.035964
    0.762942    0.037807
    0.749126    0.039619
    0.735308    0.041399
    0.721487    0.043147
    0.707665    0.044861
    0.693845    0.046545
    0.680028    0.048194
    0.666213    0.049808
    0.652403    0.051386
    0.638599    0.052928
    0.624798    0.054431
    0.611005    0.055899
    0.597219    0.057325
    0.583443    0.058711
    0.569677    0.060055
    0.555923    0.061355
    0.542181    0.062610
    0.528453    0.063818
    0.514737    0.064978
    0.501039    0.066087
    0.487357    0.067145
    0.473698    0.068151
    0.460062    0.069098
    0.446451    0.069986
    0.432864    0.070810
    0.419303    0.071570
```

```
NACA 1412
    1.000000    0.001260
    0.994709    0.002180
    0.985840    0.003708
    0.975932    0.005393
    0.965177    0.007198
    0.953813    0.009077
    0.942052    0.010991
    0.930050    0.012913
    0.917903    0.014827
    0.905675    0.016722
    0.893398    0.018594
    0.881092    0.020440
    0.868768    0.022258
    0.856431    0.024048
    0.844084    0.025809
    0.831731    0.027542
    0.819370    0.029246
    0.807006    0.030921
    0.794635    0.032567
    0.782261    0.034186
    0.769883    0.035775
    0.757501    0.037335
    0.745116    0.038867
    0.732726    0.040370
    0.720336    0.041844
    0.707942    0.043289
    0.695546    0.044705
    0.683151    0.046091
    0.670755    0.047447
    0.658361    0.048773
    0.645968    0.050066
    0.633576    0.051328
    0.621186    0.052556
    0.608798    0.053750
    0.596410    0.054910
    0.584024    0.056035
    0.571641    0.057125
    0.559259    0.058176
    0.546879    0.059193
    0.534503    0.060170
    0.522129    0.061109
    0.509759    0.062008
    0.497395    0.062868
    0.485039    0.063685
    0.472692    0.064460
    0.460356    0.065188
```

**Figure 2: Normalized Coordinate Files of NACA 0015 and 1412.**

As shown in figure 2, the x co-ordinates varied for each airfoil, thus each set of x co-ordinates had to be added to the dataset.

Using a constant panel bunching parameter which worked for complex airfoils, amongst simpler airfoils, resulted in an error state during modelling on XFoil due to the airfoils' reduced curvature, requiring a smaller panel bunching parameter. The error is shown in figure 3 represented by the purple front colour, which was resolved by tuning the panel bunching parameter to the nearest 5 degrees for optimal consistency. The number of panel-nodes was set to 300 after trial and error, which was the maximum number possible to be used for a variety of geometric airfoil variations.

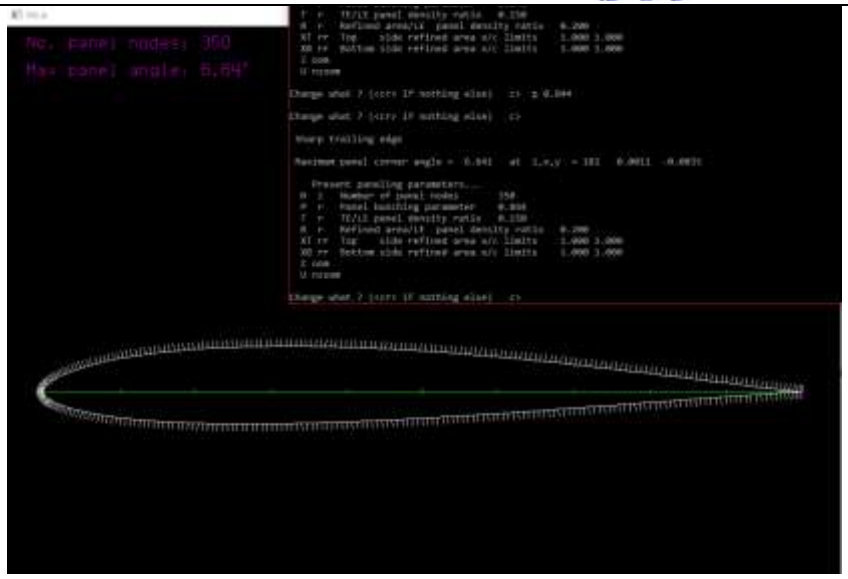



**Figure 3: XFoil Error State due to bad panel bunching parameters.**

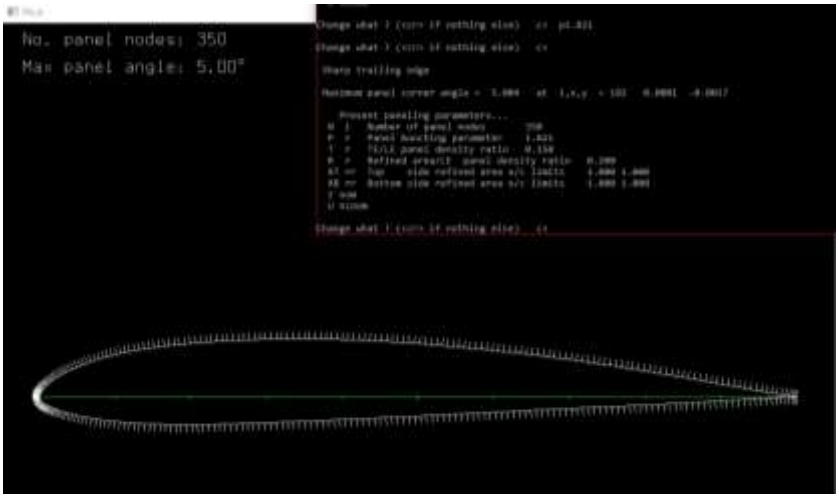


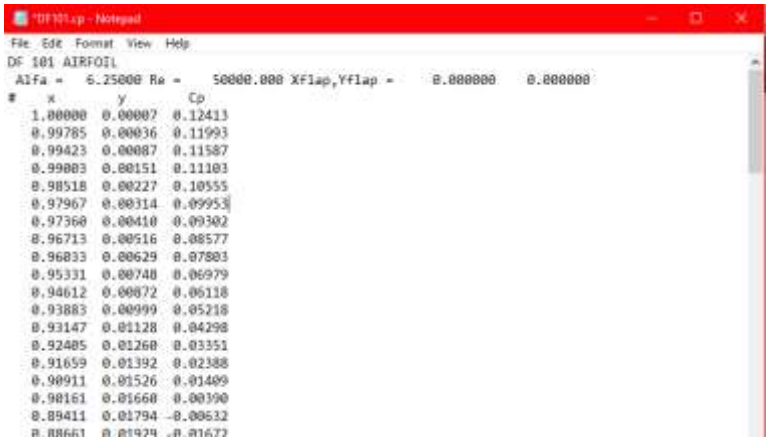

**Figure 4: XFoil Tuned Panel Bunching Parameters.**

Figure 4 shows an example of successfully tuned panelling parameters.

- The flow speed during the simulation was set to Mach 0.1 in order to understand the behaviour of low Reynolds number airfoils at low speed, due to little research in the area (Brusov et al., 2011).
- A Reynolds number of 50000 was used to set the fluid viscosity for the XFoil simulation. The Reynolds number was chosen as it was found to be the lowest commonly used Reynolds number for Earth (Winslow et al., 2018).
- Following the collection of high lift, low Reynolds number airfoils, they were organised by their optimal angle of attack, to obtain the modal angle of attack. The modal angle of attack was found to be 6.25 degrees, which was the parameter used within XFoil when performing simulations.

- The amplification factor or nCrit, was set to 9 to simulate the flow conditions of an “average wind tunnel” (airfoiltools, 2021)
- The parameters held constant throughout the simulations were the Reynolds number and the flow speed, in order to evaluate the varying lift upon the aerofoil (MIT, 2001).
- The default number of iterations for the simulation was set to 10 (MIT, 2001),
- although due to failed convergence of the boundary layer, the number of iterations was raised to 10000, which leveraged the increased computing power available at the time of writing. The simulations were then found to have no convergence errors.

XFoil’s CPWR function was used to save the pressure distribution in a tabular format as represented by figure 5, which was used as input data for the deep neural network models.



```
DF 101 AIRFOIL
 Alfa =   6.25000 Re =    50000.000 Xflap,Yflap =    0.000000    0.000000
#    x         y         Cp
   1.00000   0.00007   0.12413
   0.99785   0.00036   0.11993
   0.99423   0.00087   0.11587
   0.99003   0.00151   0.11103
   0.98518   0.00227   0.10555
   0.97967   0.00314   0.09953
   0.97360   0.00410   0.09302
   0.96713   0.00516   0.08577
   0.96033   0.00629   0.07803
   0.95331   0.00748   0.06979
   0.94612   0.00872   0.06118
   0.93883   0.00999   0.05218
   0.93147   0.01128   0.04298
   0.92405   0.01260   0.03351
   0.91659   0.01392   0.02388
   0.90911   0.01526   0.01409
   0.90161   0.01660   0.00390
   0.89411   0.01794  -0.00632
```

**Figure 5: The output file from XFoil’s CPWR function.**

Upon opening the file using Python, the data was presented in a string format. Using the Pandas library (Pandas, 2021), the fixed width formatting technique was used to manually set the dimensions of each column, ignoring the first two lines, allowing the airfoil data to be parsed without errors. Matplotlib (2021), was used to produce a plot of each airfoil, using its co-ordinate data. The plot used the pressure coefficients of the airfoil as the hue to visualize the pressure distribution of the data, as shown in figure 6. The process was undertaken to verify the parsing of the airfoil data.

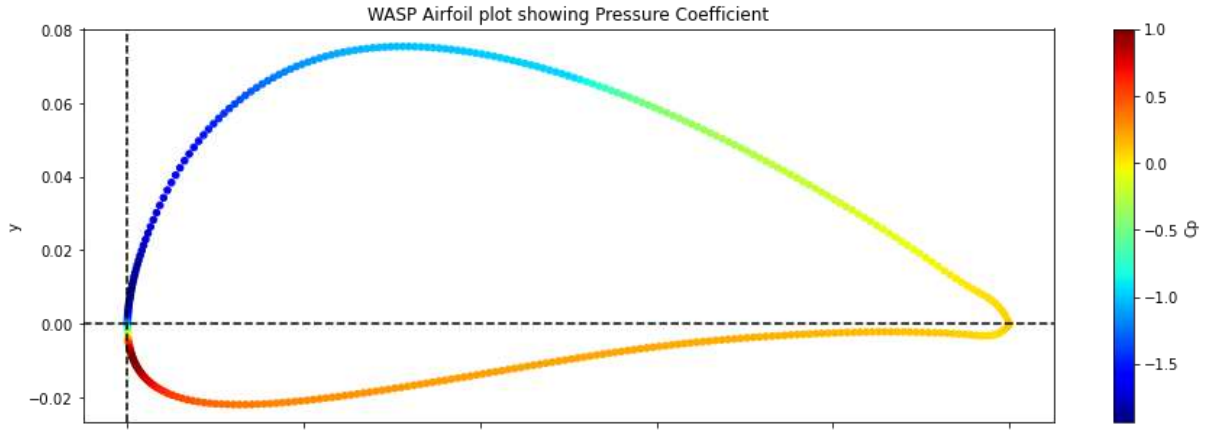



**Figure 5: WASP Airfoil Pressure Distribution Plot.**

Once the parsing of the data was verified, the pre-processed airfoil data was saved as a pickle file, for use in the final data frame. The pre-processed airfoil data was appended to a single data frame using Pandas (2021), and then its rows were shuffled to improve the effectiveness of the training of the model. After previewing the completed data frame, as shown in figure 6, the final data frame was saved as a pickle file, for use in the deep neural network model.

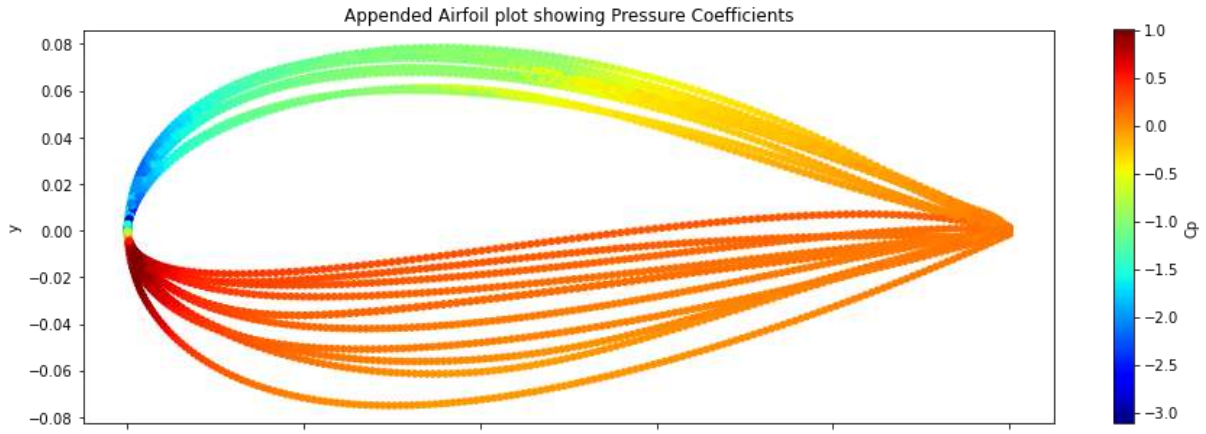



**Figure 6: Appended Data Frame Airfoil Pressure Distribution Plot**

A count plot (Seaborn, 2021), shown in figure 7, was used to evaluate the distribution of the categorical variables, to evaluate whether the diversity of the data frame was sufficient. The distribution of the data was inspected once again, as shown in figures 8 and 9, following the splitting of the training data from the test data, which was split at a 4:1 ratio, respectfully. A correlation heatmap (Pandas, 2021), shown in figure 10, was also produced to evaluate the effect of each feature set, upon the label set.

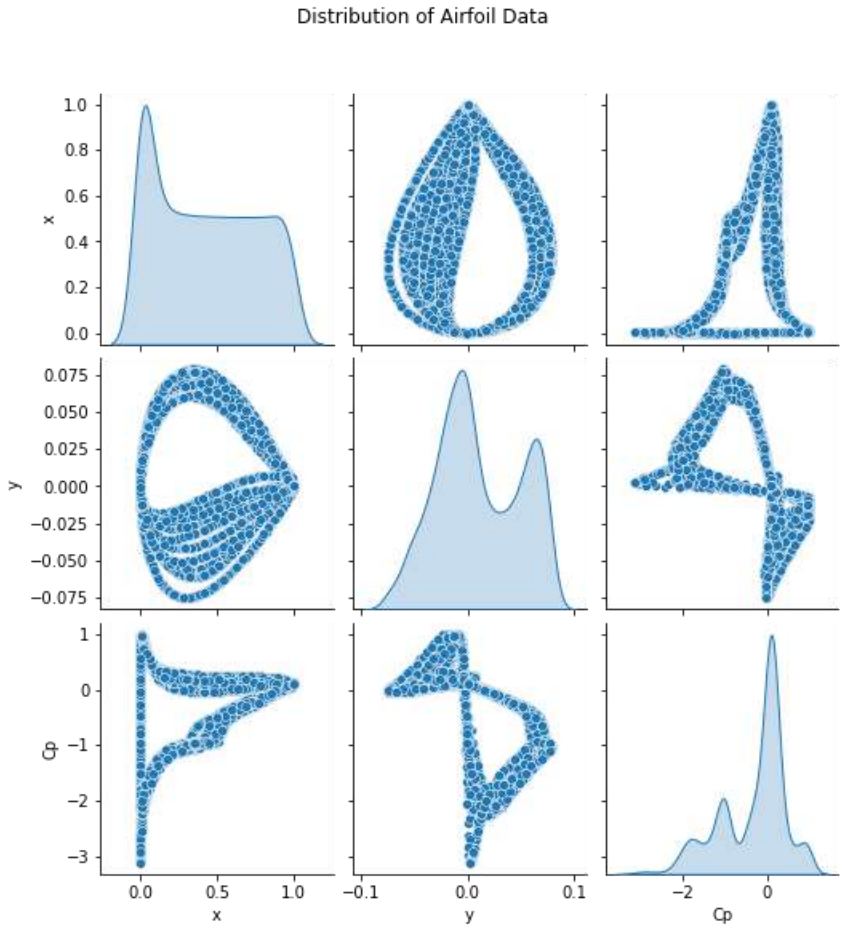



**Figure 7: Data Frame Count Plot.**

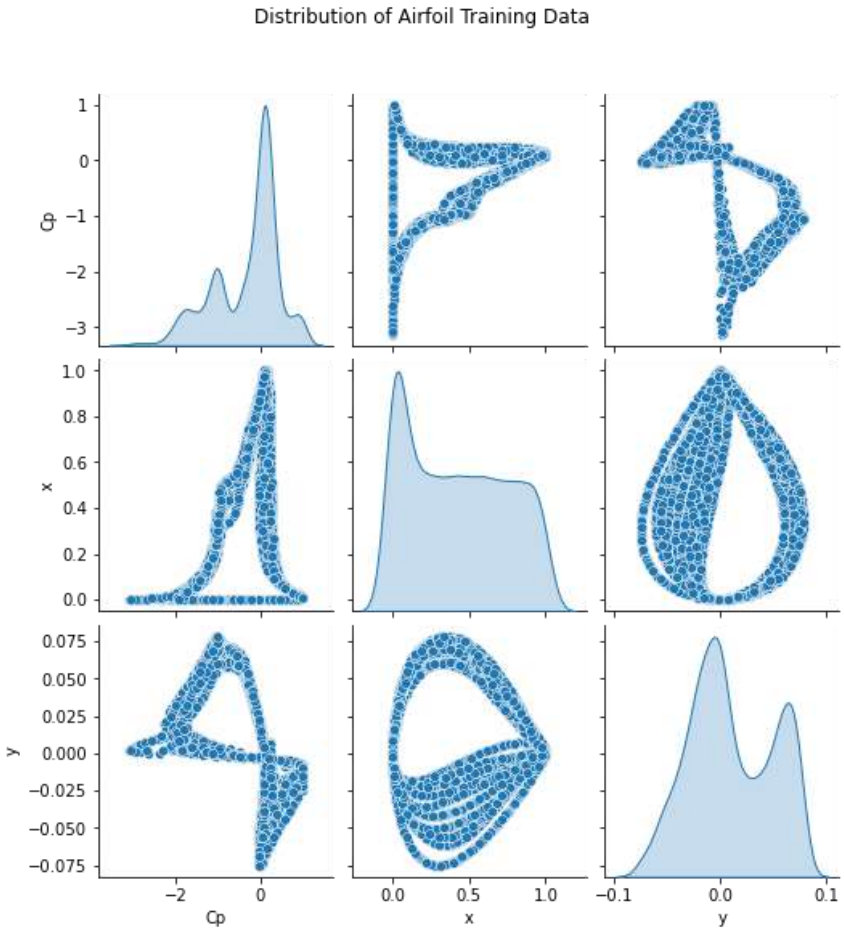



**Figure 8: Training Data Count Plot.**

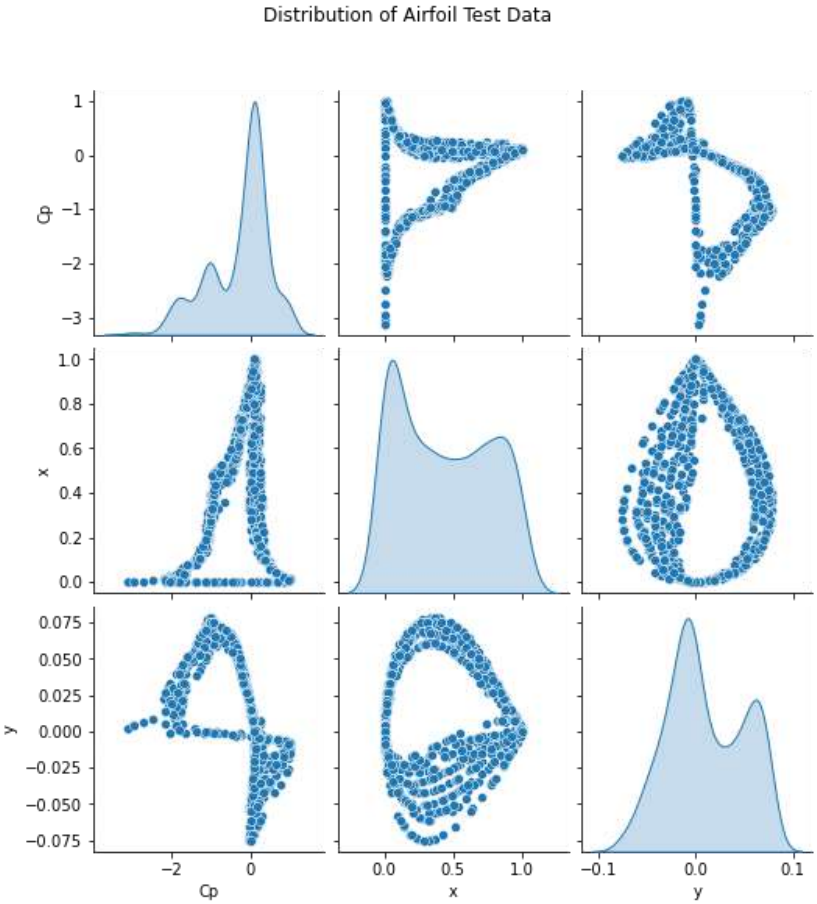



**Figure 9: Test Data Count Plot.**

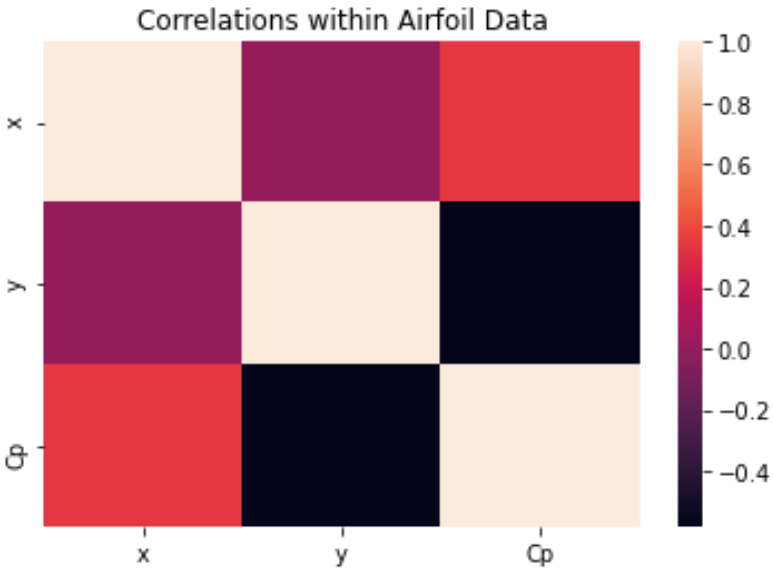



**Figure 10: Correlation Heatmap of the Data Frame**

The mean and standard deviation of the data frame was assessed to ensure there were no anomalies. From table 2, it was found that there were no extreme values, reassuring a lack of anomalies within the data frame.

| Variable | Mean | Standard Deviation |
|---|---|---|
| **x** | 0.425563 | 0.323191 |
| **y** | 0.010099 | 0.039288 |
| **Cp** | -0.328002 | 0.770859 |

**Table 2: Mean and Standard Deviation of the Data Frame.**

By normalizing the data, the feature sets and label set are scaled to values between 0 and 1, in order to improve the performance of the deep neural network.

The effect of changing the number of neurons on the error of a single layer deep neural network model, was investigated. It was found that by optimizing the number of neurons within the hidden layer for the data set, underfitting or overfitting could be optimized. The number of neurons used in neural networks go up by the power of 2, as represented by table 3.

| | Mean absolute error | |
|---|---|---|
| **Number of Nodes** | **Linear Model** | **Deep Neural Network Model** |
| **32** | 0.529227 | 0.32327 |
| **64** | 0.534267 | 0.308138 |
| **128** | 0.509083 | 0.293942 |
| **256** | 0.50772 | 0.269882 |
| **512** | 0.520505 | 0.280725 |
| **1024** | 0.512693 | 0.295765 |

**Table 3: Mean absolute error for Linear and Deep Neural Network Models.**

As shown in table 3, that the deep neural network model had significantly more accuracy than the linear, control model. The data also showed that it was not possible to fit the data to a linear model, due to its poor results. The optimal number of nodes for the deep neural network was found to be 256 neurons, which represented the

lowest mean absolute error. As a result of the evaluation, the maximum number of neurons per layer used for testing was 256 neurons.

As shown in table 4. one to eight layer neural networks were considered, with The table shows the neural networks that were tested for use in the prediction model. The number of layers mentioned in the table do not include the input and output layers.

| Deep Neural Network Type |
|---|
| **Linear Model.** |
| **One Layer 256 Neuron.** |
| **Two Layer, 256 Neuron to 128 Neuron.** |
| **Four Layer 16 Neuron to 2 Neuron.** |
| **Four Layer 128 Neuron to 16 Neuron.** |
| **Four Layer 256 Neuron to 32 Neuron.** |
| **Eight Layer 256 Neuron to 2 Neuron.** |

**Table 4: Neural Networks to be tested.**

Fitting of data was found to be optimized when the number of hidden layers in the deep neural network were adjusted. While linear models were found to have no hidden layers, they only produced accurate results with linear data. In the case of the airfoil data, a non-linear relationship was observed. The effect of increasing the number of hidden layers was evaluated during model selection. It was found that using the leaky rectified linear unit activation, using less than 2 hidden layers produced an underfitting model, while using more than 2 hidden layers resulted in overfitting. When the exponential linear unit activation was used, 4 layers were found to be the optimal configuration.

Following extensive research to identify the appropriateness of the application, it was found that there were limited activation functions available to interpret negative values. There was no previous research on the use of the exponential linear unit activation for the use of airfoil pressure distribution prediction, although the deep neural network model by Bhatnagar et al. (2019), had used the rectified linear unit activation. An alternative configuration of the rectified linear unit activation, known as the leaky rectified linear unit activation, was used for initial testing. The leaky rectified linear unit

activation allows a set leakage of the $y$ intercept, alpha (α) from the originally defined 0 of a rectified linear unit activation , to a value of 0.01, to prevent the "vanishing gradient" (Géron, 2019) problem experienced when using the rectified linear unit activation. The leaky rectified linear unit activation used the same linear relationship for positive values $0 >$ y, and sharply transitioned to a much lower gradient from $-0.01 > y > 0$ The fitting of the model showed further improvements when using the exponential linear unit activation, which used negative input values of $-1 > y > 0$. The exponential linear unit activation was found to have used an exponential slope from $-1 > y > 0$, and followed a linear relationship similar to the rectified linear unit activation for positive values $0 >$ y. The activations were represented in graphical form by figure 11 (Shenoy, 2019).

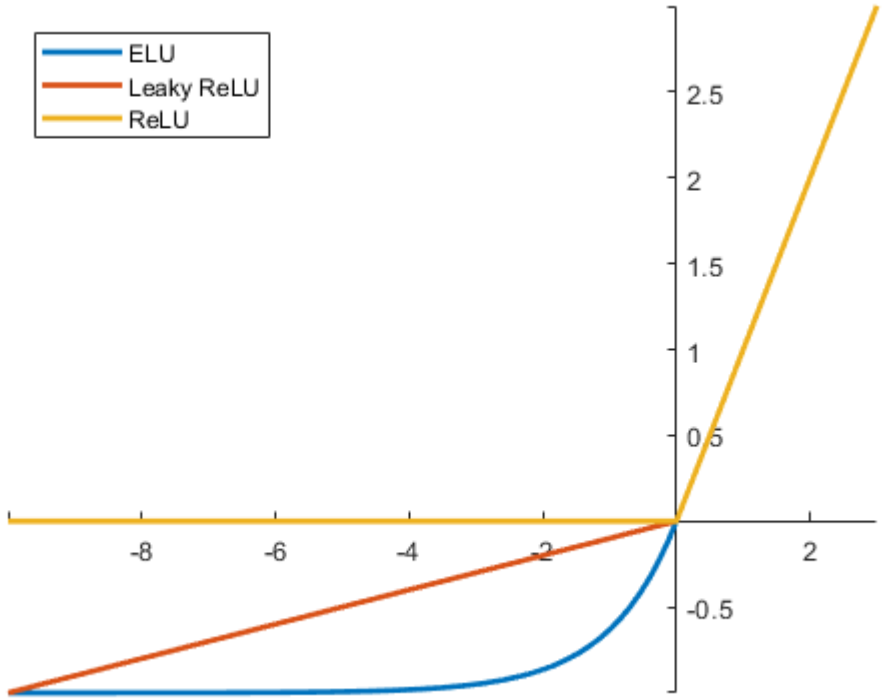



**Figure 11: Graphical Comparison of Neural Network Activations (Shenoy, 2019).**

For the aforementioned reason, the rectified linear unit activation was used for initial testing, to verify that the model would a produce similar accuracy score to the convolutional neural network model by Bhatnagar et al. (2019). Once a suitable configuration was established for the initial deep neural networks, the rectified linear unit activation was substituted for the exponential linear unit activation. The benefit of

using the exponential linear unit activation was found to be its increase of convergence when $y < 0$, which was found to be important for the deep neural network to accurately visualize the co-ordinate points, resulting in more accurate predictions.

The adamax (Keras 2021) optimizer was found to be the optimal optimizer to use. The optimizers compared were adam (Keras 2021), adamax and RMSprop (Keras 2021). Although RMSprop (Keras 2021) produced similar prediction accuracy to adamax (Keras 2021), a longer training time was required. While adam (Keras 2021) was a second order algorithm, adamax (Keras 2021) was an infinite order algorithm, and was found to produce the least noise during training, as it used gradient boosting (Géron, 2019), resulting in fewer training losses. Figure 12 (Tahmassebi, 2018) was found to represent the training and testing accuracy of various optimization algorithms, and had shown that adamax (Keras 2021) was the most accurate solution.

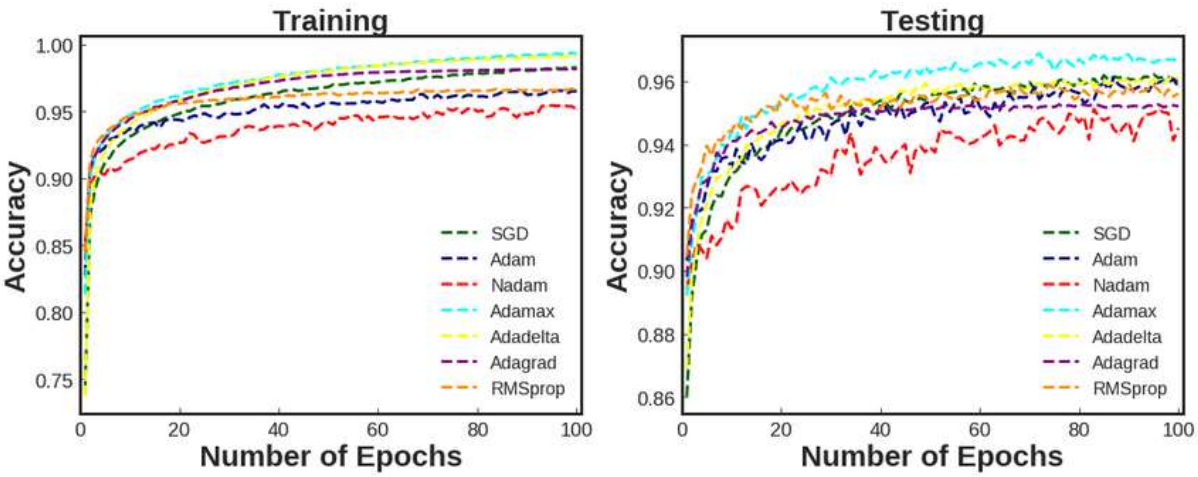



**Figure 12: Graphical Comparison of Neural Network Optimizer Accuracies (Tahmassebi, 2018).**

The loss function defined the equation the deep neural network used to visualize its accuracy, during each epoch of training. When the neural network was able to visualize each epoch effectively, it was found to reduce training loss, while reducing the epochs required to train the model (Géron, 2019). When the options for the loss function were reviewed, it was found that there were “three commonly used” (Brownlee, 2021) regression metrics, which were listed below:

- Mean Squared Error.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \widehat{Y}_i)^2$$

Where $n$ is the sample size, $Y_i$ is the observed value and $\widehat{Y}_i$ is the predicted value.

Mean squared error was found to be a metric for determining how close a fitted line is to the real data points. Although the mean squared error can provide a metric to compare against other models, it cannot be interpreted in terms of measurement units, which in the case of the completed project, was the pressure coefficient, Cp.

- Root Mean Squared Error.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(Y_i - \widehat{Y}_i)^2}$$

The definition of the variables for root mean squared error are the same as mean squared error.

Root mean squared error takes the square root of the mean squared error, providing a value of error in the units of the observed value, the pressure coefficient, Cp.

- Mean Absolute Error.

$$MAE - \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

The mean absolute error was found to be the average of the absolute differences between the predictions and actual observations throughout the test sample, which had an equal weight assigned to all individual deviations (Wesner, 2020). As shown by figure x (Wesner, 2020), the mean absolute error hadn't changed when a large outlier was found, while the root mean squared error was found to reflect the anomaly. Wesner (2020), had found, as shown in figure 13, that when using the root mean squared error as the loss function, the optimal accuracy was observed.

As the model was not compatible with the root mean squared error loss function, the mean squared error loss function was used, as the loss function was used for the deep neural network and not for evaluation.

CASE 1: Evenly distributed errors

| ID | Error | \|Error\| | Error^2 |
|---|---|---|---|
| 1 | 2 | 2 | 4 |
| 2 | 2 | 2 | 4 |
| 3 | 2 | 2 | 4 |
| 4 | 2 | 2 | 4 |
| 5 | 2 | 2 | 4 |
| 6 | 2 | 2 | 4 |
| 7 | 2 | 2 | 4 |
| 8 | 2 | 2 | 4 |
| 9 | 2 | 2 | 4 |
| 10 | 2 | 2 | 4 |

| MAE | RMSE |
|---|---|
| 2.000 | 2.000 |

CASE 2: Small variance in errors

| ID | Error | \|Error\| | Error^2 |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
| 2 | 1 | 1 | 1 |
| 3 | 1 | 1 | 1 |
| 4 | 1 | 1 | 1 |
| 5 | 1 | 1 | 1 |
| 6 | 3 | 3 | 9 |
| 7 | 3 | 3 | 9 |
| 8 | 3 | 3 | 9 |
| 9 | 3 | 3 | 9 |
| 10 | 3 | 3 | 9 |

| MAE | RMSE |
|---|---|
| 2.000 | 2.236 |

CASE 3: Large error outlier

| ID | Error | \|Error\| | Error^2 |
|---|---|---|---|
| 1 | 0 | 0 | 0 |
| 2 | 0 | 0 | 0 |
| 3 | 0 | 0 | 0 |
| 4 | 0 | 0 | 0 |
| 5 | 0 | 0 | 0 |
| 6 | 0 | 0 | 0 |
| 7 | 0 | 0 | 0 |
| 8 | 0 | 0 | 0 |
| 9 | 0 | 0 | 0 |
| 10 | 20 | 20 | 400 |

| MAE | RMSE |
|---|---|
| 2.000 | 6.325 |

**Figure 13: Comparison of the Performance of Common Loss Functions (Wesner, 2020).**

Due to the inability to view the mean squared error during training, the deep neural network models were evaluated using a root squared ($R^2$) score. The root squared score was obtained using the equation shown, where $\mu$ was the residual of the predicted values against the observed values (Géron, 2019).

$$R^2 = 1 - \frac{\sum_{i=1}(y_i - \hat{y}_i)^2}{\sum_{i=1}(y_i - \mu)^2}$$

The root squared score produced a value of a similar dimension to an accuracy score, the closer the value was to 1, the greater the accuracy of the model was found to be.

Loss plots were produced during the training of the deep neural network models to ensure the training and testing losses could be visualized. The loss plots were used to estimate the optimal number of epochs required for training, as well as to evaluate the noise produced during the training of each model.

The convergence of the predictions to the test data were evaluated to visualize the effectiveness of the deep neural network. The prediction plots were used to find anomalies within the prediction data, such as groups of outliers, in order to diagnose errors encountered by the deep neural network.

Error plots of the deep neural network model were produced in order to find the modal source of errors when predictions were made using the deep neural networks. The source of the errors were highlighted to diagnose the root cause of any issues.

When predictions were made, the computation times were recorded using the iPython line magic time function. The computation times, recorded in milliseconds, were displayed when "%%time" (iPython, 2021) was placed before the execution of predictions for the models.

# 6 Results and Analysis.

The parametrization techniques discussed in the design development section were followed to obtain simulations of the pressure distribution of each airfoil, in viscous conditions. Figures 5 shows results of the simulation of the DF101 airfoil. The simulation results of the remainder of the used airfoils were located in appendix B. Appendix B shows all airfoils had converged successfully.

From figures 7 to 10 of the data visualization process, the distribution of the data was found to be satisfactory, although the pressure coefficient data was skewed toward zero, which could not be remedied as the airfoils had a large area of a balanced pressure distribution. The x co-ordinates showed a strong positive correlation with the pressure coefficient data, while the y co-ordinates showed a strong negative correlation with the pressure coefficient data.

From the loss plot data shown in appendix C, it was found that when a larger number of layers were used within the neural network, much fewer epochs took place before the deep neural network had stabilized, vice versa. As the number of neurons within the deep neural networks increased, the noise within the neural networks increased, as represented by the increased amplitude of the training and validation loss lines, on the loss plot.

The deep neural network evaluation used two activation functions for testing, The leaky rectified linear unit and the exponential linear unit, as aforementioned. The deep neural network evaluation results were converted to tabular form, as shown in tables 5 and 6.

| Exponential Linear Unit Neural Network Type | MSE [Cp] | $R^2$ Score (4 d.p) |
|---|---|---|
| **Fourlayer_128to16_dnn** | 0.014328 | 0.9758 |
| **Fourlayer_256to32_dnn** | 0.014602 | 0.9753 |
| **Eightlayer_256to2_dnn** | 0.014802 | 0.9750 |
| **Twolayer_256to128_dnn** | 0.016132 | 0.9727 |
| **Onelayer_r256_dnn** | 0.038236 | 0.94 |
| **Linear_model (Tensorflow, 2021)** | 0.327267 | 0.4480 |
| **Fourlayer16to2_dnn** | 0.560118 | 0.0552 |

**Table 5: ELU Deep Neural Network Performance Evaluation.**

| Leaky Rectified Linear Unit Neural Network Type | MSE [Cp] | $R^2$ Score (4 d.p) |
|---|---|---|
| **Twolayer_256to128_dnn** | 0.014601 | 0.9753 |
| **Fourlayer_128to16_dnn** | 0.014628 | 0.9746 |
| **Onelayer_r256_dnn** | 0.015736 | 0.9746 |
| **Fourlayer_256to32_dnn** | 0.018065 | 0.97 |
| **Eightlayer_256to2_dnn** | 0.018439 | 0.9689 |
| **Linear_model (Tensorflow, 2021)** | 0.327209 | 0.4481 |
| **fourlayer16to2_dnn** | 0.407165 | -0.1315 |

**Table 6: RELU Deep Neural Network Performance Evaluation.**

From tables 5 and 6, it was found that the four layer, 128 neuron to 16 neuron exponential linear unit deep neural network was the optimal configuration from the tested models. The second best model was the four layer, 256 neuron to 32 neuron exponential linear unit deep neural network, followed by the two layer, 256 neuron to 128 neuron leaky rectified linear unit deep neural network.

From the prediction plot data shown in appendix D, it was found that the optimal configuration for the best convergence was when larger numbers of layers were used. The eight layer deep neural network showed evidence of good fitting although it was reflected by its root squared score that overfitting had taken place. It was found that below a pressure coefficient of -1, there was no longer convergence of the model. When the exponential linear unit activation models were used, the lack of convergence appeared at a lower pressure coefficient value than with the leaky rectified linear unit activation, which proved that the problem required an algorithm compatible with negative input values.

Amongst the error plot data shown in appendix E, a large number of errors were observed near to where the pressure coefficient was near to zero. The number of layers within the neural network was not found to have an effect on the error count.

| Neural Network Type | Time (ms) |
|---|---|
| **fourlayer_128to16_ELU_dnn** | 144 |
| **fourlayer_256to32_ELU_dnn** | 150 |
| **eight_layer_256to2_ELU_dnn** | 150 |
| **twolayer_256to128_ReLU_dnn** | 181 |
| **fourlayer_128to16_ReLU_dnn** | 183 |
| **onelayer_256_ReLU_dnn** | 223 |

**Table 7: Model Predictions Speed Evaluation.**

From table 7, it was found that the deep neural network model with the highest accuracy also took the least time to execute its predictions. Meanwhile, all deep neural networks that used exponential linear unit activation functions performed at least 20% faster than deep neural networks which used leaky rectified linear unit activation.

# 7 Discussion.

Although the waterfall project management methodology was found to be straightforward, when adjustments had to be made to previous stages of the model, the structure of the waterfall project management methodology were ignored. Due to the fact the deep neural network model required continuous improvement, it was not possible to follow the structure of the waterfall project management methodology as planned.

During model evaluation, the large count of errors when the pressure coefficient was equal to zero was found to be due to a large concentration of co-ordinate points at the tip of the airfoils, resulting in various values for the lift coefficient to be represented by the same co-ordinate points. With the exponential linear unit deep neural network unable to interpret time steps prior to and after its current time step, the exponential linear unit deep neural network was found to deliver poor accuracy for values at the beginning and the end of its co-ordinate values.

Compared to the initially researched convolutional neural network (Zelong et al., 2019) which was used to evaluate the pressure distribution of 2D airfoils, the four layer 128 neuron to 16 neuron exponential linear unit deep neural network had shown a 50% improvement in accuracy. The reduction in model complexity was shown to have provided more accurate results although the regression deep neural network was not compatible with varying angles of attack or flow speed, unlike the model designed by Zelong et al. (2019).

As represented by figure 1, the project was 95% complete. The remaining tasks to complete the project were to undertake hyperparameter tuning using KerasTuner (Keras, 2021), and to deploy the chosen deep neural network to the Microsoft Azure cloud server (Microsoft, 2021). The aforementioned aspects of the project were not completed due to a lack of time. Hyperparameter tuning of the four layer 128 neuron to 16 neuron deep neural network would improve training times by optimizing the number of epochs required for training, as well as tuning the alpha value of the exponential linear unit activation, by testing a range of values within a given range, and returning the optimal parameters to be used. Deployment of the deep neural network to a cloud server such as Microsoft Azure (Microsoft, 2021), would allow the deep neural network to utilize more computing power during training, providing the deep neural network with the ability to be scaled for much larger datasets, with much shorter training and prediction times, therefore allowing the full data set of 100 airfoils to be used. As well as reduced processing times, using cloud servers also provides the benefit of accessing the deep neural network remotely, which may be useful when dealing with contingencies that result in having to leave the lab. In addition to this, the solution would allow multiple users to be able to use the latest version of the model simultaneously.

# 8 Conclusions.

In order to design a deep neural network to predict the pressure coefficients of 2D airfoils, the airfoil co-ordinate data was successfully used to produce smoothed airfoils, which were used to successfully obtain the pressure distribution models, by simulation using XFoil. The XFoil string data was converted and visualized as a Pandas (Pandas, 2021) data frame effectively. Finally, the deep neural network models were designed, compiled, trained, tested and validated. It was ensured that all evaluation metrics were

considered when it was decided that the four layer 128 neuron to 16 neuron exponential linear unit deep neural network was found to be the most effective configuration for predicting the pressure coefficient of low Reynolds number 2D airfoils. Although the model was selected, the aim of deploying the model was not achieved. Only 10% of the initially planned dataset was used for model evaluation due to limited computing power during testing, although the remainder of the data was to be used after deployment to the Microsoft Azure cloud server (Microsoft, 2021). With a 50% improvement of performance from the initially studied model, the four layer 128 neuron to 16 neuron exponential linear unit deep neural network was found to be the most effective configuration for predicting the pressure coefficient of 2D airfoils. Although the optimal model had an accuracy of 97.58% and the lowest prediction time of 144 milliseconds, it suffered convergence issues when the pressure coefficient was less than -1. The root cause of the convergence issue was found to be the deep neural network model's lack of time series awareness.

# 9 Recommendations.

In order to work more effectively, while sticking to the project plan, an agile project management methodology is recommended. Using the agile project management methodology allows for continuous improvement, without breaking the workflow. As shown in figure 14, once testing takes place, analysis takes place once again, to evaluate the effect of the changes made, before a new problem is defined and changes are made to the initial design, which are tested once again to analyse their results. Only when the developer is completely satisfied with the entire model, can the decision be made to deploy the model. As shown in the waterfall project management methodology, the production stages follow a parent-child relationship which resulted in too much time being spent dwelling on intricacies of the project, rather than the entirety to the project. The agile project management methodology allows for prototypes to be created much sooner than the waterfall methodology for the aforementioned reasons, and will result in a smoother workload throughout the project.

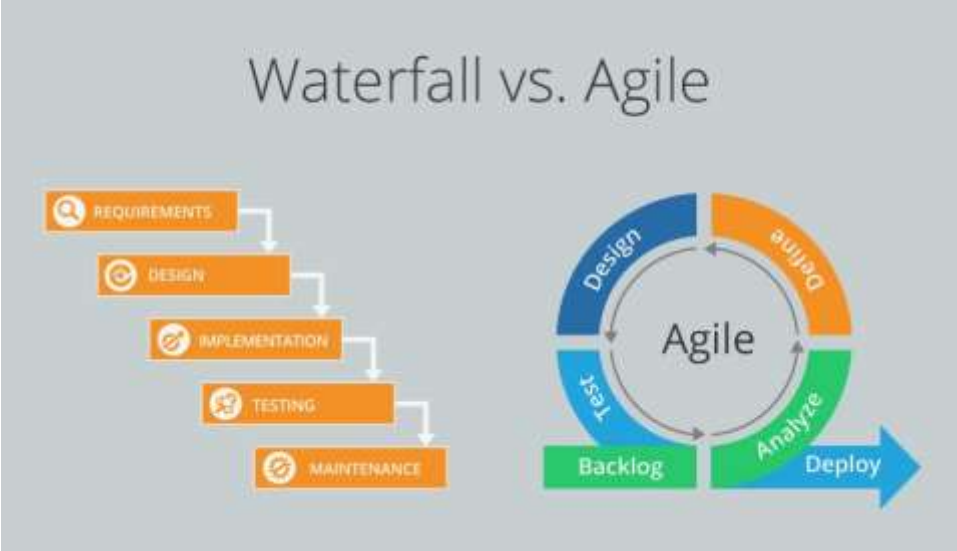



**Figure 14: Waterfall vs. Agile. (Techguide, 2020).**

To streamline the smoothing and parametrization of the airfoil co ordinate data in order to obtain simulations, it is recommended that a script is written to automatically obtain pressure coefficient data for each airfoil in the root folder, rather than the user spending up to 30 minutes per airfoil, to complete the same task. To streamline the pre-processing of the data, it is recommended that the pre-processing script is made to loop for all files within its root folder, in order to reduce the time taken individually selecting the airfoil data to be converted to a Pandas (Pandas, 2021) data frame. To streamline the production of the data frame it is recommended that the pre processed airfoil data is automatically appended to a single data frame, which collates all the

pickle files of the airfoil data in the root folder. It is recommended that a wrapper is used to enclose the pre-processing script and data frame script into a single command, to reduce the time taken to execute the model. As the optimal configuration for the deep neural network was found, the script would be more streamlined if the model compilation of the deep neural network was placed in a wrapper, so that the model can be called with a single command rather than reparametrizing the entire model. The long short term memory (Géron, 2019) deep neural network architecture allows a deep neural network to view 5 timesteps ahead of its current step to evaluate the direction of the gradient more accurately. By utilizing the architecture, the large error count at zero can be eliminated due to the deep neural network having the ability to differentiate between the various pressure coefficient values when the $y$ co-ordinate is equal to zero. By using a long short term memory (Géron, 2019) configuration, the accuracy of the model should improve dramatically. As various optimizers and loss functions were evaluated, each version of the script could have been saved to assess the development of the script, from design to conception. By using a version history, older versions can be referred to if re-evaluating a configuration under different parameters, without having to revert back to an older configuration entirely. Using a version history can also save time if mistakes were made during an update. The version history of a software application can be saved using an online storage facility called GitHub (GitHub, 2020), which allows all users of the program to be able to view the version history, as well as download older versions.. The deep neural network solution is expected to behave with increased accuracy with no anomalous values if the aforementioned changes were made.

# 10 References.

**Books**

Géron, A., 2019. *Hands-on machine learning with Scikit-Learn and TensorFlow*. Sebastopol (Clif.) [etc.]: O'Reilly.

**Research papers.**

Deters, R., Selig, M. and Ananda, G., 2014. *Reynolds Number Effects on the Performance of Small-Scale Propellers*. [online] M-selig.ae.illinois.edu. Available at: <https://m-selig.ae.illinois.edu/pubs/DetersAnandaSelig-2014-AIAA-2014-2151.pdf> [Accessed 14 June 2021].

Désert, T., Moschetta, J. and Bézard, H., 2017. Aerodynamic Design of a Martian Micro Air Vehicle. *EUCASS*,.

Levchenko, I., Xu, S., Mazouffre, S., Keidar, M. and Bazaka, K., 2018. Mars Colonization: Beyond Getting There. *Global Challenges*, 3(1), p.1800062.

Bhatnagar, S., Afshar, Y., Pan, S., Duraisamy, K. and Kaushik, S., 2019. Prediction of aerodynamic flow fields using convolutional neural networks. *Computational Mechanics*, 64(2), pp.525-545.

Winslow, J., Otsuka, H., Govindarajan, B. and Chopra, I., 2018. Basic Understanding of Airfoil Characteristics at Low Reynolds Numbers (104–105). *Journal of Aircraft*, 55(3), pp.1050-1061.

Levchenko, I., Xu, S., Mazouffre, S., Keidar, M. and Bazaka, K., 2018. Mars Colonization: Beyond Getting There. *Global Challenges*, 3(1), p.1800062.

Morgado, J., Vizinho, R., Silvestre, M. and Páscoa, J., 2016. XFOIL vs CFD performance predictions for high lift low Reynolds number airfoils. *Aerospace Science and Technology*, 52, pp.207-214.

Brusov, V. and Petruchik, V., 2011. Design Approach for Selection of Wing Airfoil with Regard to Micro-UAVs. *IMAV 2011*, [online] Available at: <https://doi.org/10.4233/uuid:2b261a19-fae4-44dd-ac04-1113ebc7c89b> [Accessed 14 June 2021].

Zelong, Y., Wang, Y., Qiu, Y., Bai, J. and Chen, G 2018. Multiple Aerodynamic Coefficient Prediction of Airfoils Using a Convolutional Neural Network. *Symmetry*, 12(4), p.544.

Ward, C. and Legorreta, L., 2009. Beyond Waterfall and Agile Methods: Towards a New Contingency Model for IT Project Management. *SSRN Electronic Journal*,.

Chen, T., 2009. *Artificial Neural Network Investigation of Aerodynamic Pressure Load Modelling*. [online] Academia.edu. Available at: <https://www.academia.edu/7670889/Artificial_Neural_Network_Investigation_of_Aerodynamic_Pressure_Load_Modelling> [Accessed 14 June 2021].

Shenoy, A., 2019. *(PDF) Feature optimization of contact map predictions based on inter-residue distances and U-Net++ architecture*. [online] ResearchGate. Available at: <https://www.researchgate.net/publication/334389306_Feature_optimization_of_contact_map_predictions_based_on_inter-residue_distances_and_U-Net_architecture> [Accessed 14 June 2021].

Tahmassebi, A., 2018. *(PDF) iDeepLe: deep learning in a flash*. [online] ResearchGate. Available at: <https://www.researchgate.net/publication/325055629_iDeepLe_deep_learning_in_a_flash> [Accessed 14 June 2021].

**Websites**

NASA, 2021. *Earth Fact Sheet*. [online] Nssdc.gsfc.nasa.gov. Available at: <https://nssdc.gsfc.nasa.gov/planetary/factsheet/earthfact.html> [Accessed 14 June 2021].

NASA, 2021. *Mars Fact Sheet*. [online] Nssdc.gsfc.nasa.gov. Available at: <https://nssdc.gsfc.nasa.gov/planetary/factsheet/marsfact.html> [Accessed 14 June 2021].

Keras, 2021. *Keras: the Python deep learning API*. [online] Keras.io. Available at: <https://keras.io/> [Accessed 14 June 2021].

Pandas, 2021. *pandas - Python Data Analysis Library*. [online] Pandas.pydata.org. Available at: <https://pandas.pydata.org/> [Accessed 14 June 2021].

Airfoiltools, 2021. *Airfoil Tools*. [online] Airfoiltools.com. Available at: <http://airfoiltools.com/> [Accessed 14 June 2021].

Microsoft, 2021. *Cloud Computing Services | Microsoft Azure*. [online] Azure.microsoft.com. Available at: <https://azure.microsoft.com/> [Accessed 14 June 2021].

MIT, 2001. [online] Web.mit.edu. Available at: <http://web.mit.edu/aeroutil_v1.0/xfoil_doc.txt> [Accessed 14 June 2021].

Matplotlib, 2021. *Matplotlib: Python plotting*. [online] Matplotlib.org. Available at: <https://matplotlib.org/> [Accessed 14 June 2021].

Seaborn, 2021. *seaborn: statistical data visualization — seaborn 0.11.1 documentation*. [online] Seaborn.pydata.org. Available at: <https://seaborn.pydata.org/> [Accessed 14 June 2021].

TechGuide, 2020. *Why Agile is So Beneficial for Tech Development Companies - Tech Guide*. [online] Tech Guide. Available at: <https://www.techguide.com.au/news/internet-news/agile-beneficial-tech-development-companies/> [Accessed 14 June 2021].

GitHub, 2021. *GitHub: Where the world builds software*. [online] GitHub. Available at: <https://github.com/> [Accessed 14 June 2021].

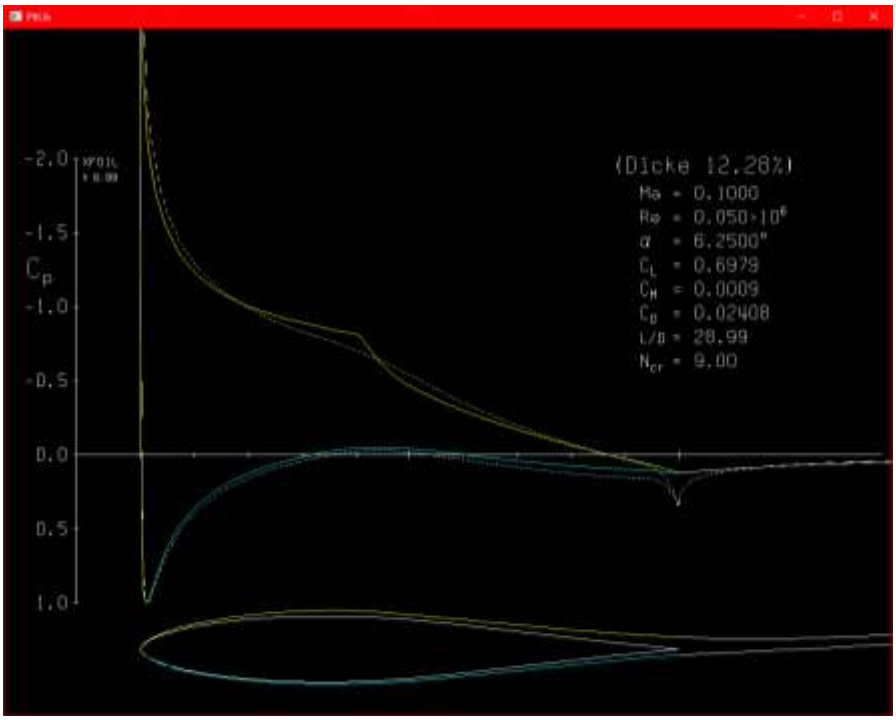
# 11 Appendix.

| Airfoil | Cl/Cd | α |
|---|---|---|
| Selig S1223 | 33.1 | 2 |
| Dillner 20–32-C | 31.7 | 2.25 |
| Selig S1210 | 14.9 | 2.5 |
| Eppler E169 | 16.7 | 3 |
| NACA 0006 | 24.9 | 3.5 |
| Selig S4083 | 31.1 | 3.75 |
| NACA 0008 | 26.1 | 4 |
| NACA 0009 | 27 | 4 |
| NACA 0012 | 20.3 | 4.25 |
| NACA 0010 | 25.9 | 4.5 |
| Selig/Donovan SD8020 | 26.1 | 4.5 |
| Selig S8025 | 22.7 | 4.75 |
| NACA 1408 | 32.3 | 4.75 |
| NACA 0024 | 15.4 | 5 |
| NACA 0021 | 19.1 | 5 |
| Selig/Donovan SD7003 | 29.3 | 5 |
| David Fraser DF102 | 32.7 | 5 |
| Eppler E230 | 23 | 5.25 |
| Deflt University DU 86-084/18 | 32.3 | 5.25 |
| Selig/Donovan SD8000 | 33.6 | 5.25 |
| Selig S6062 | 34.7 | 5.25 |
| NACA 1410 | 31.2 | 5.5 |
| Selig S2050 | 33.2 | 5.5 |
| Selig S2055 | 35.1 | 5.5 |
| Selig S2062 | 36 | 5.5 |
| Selig S2060 | 36.4 | 5.5 |
| Selig S2048 | 37.2 | 5.5 |
| NACA 2408 | 37.4 | 5.5 |
| Eppler E59 | 43.1 | 5.5 |
| NACA 0018 | 22.1 | 5.75 |
| Eppler E228 | 27 | 5.75 |
| LRN1007 | 30.6 | 5.75 |
| Selig S6062 | 34.6 | 5.75 |
| Selig/Donovan SD2030 | 37.6 | 5.75 |
| Selig/Donovan SD7090 | 30.7 | 6 |
| Selig/Donovan SD7084 | 31.3 | 6 |
| Selig/Donovan SD5060 | 33.6 | 6 |
| Selig S9000 | 34.7 | 6 |
| Rolf Girsberger RG 15 | 34.8 | 6 |
| Selig S2046 | 38.7 | 6 |
| Selig/Ashok SA7024 | 38.8 | 6 |

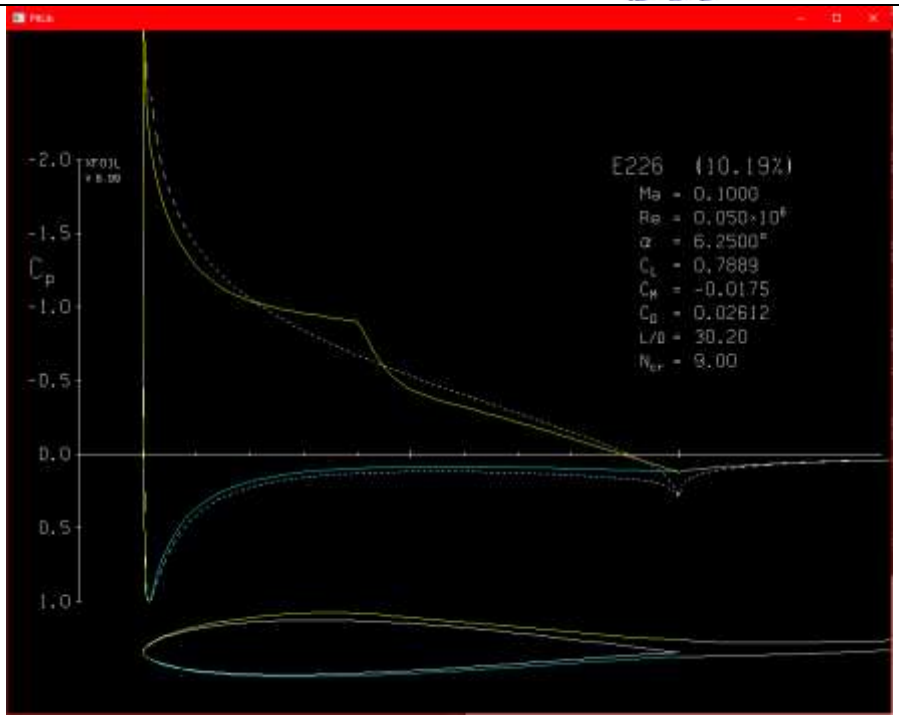
| | | |
|---|---|---|
| Selig S4110 | 39.2 | 6 |
| NACA 0015 | 24.7 | 6.25 |
| Eppler E171 | 29.2 | 6.25 |
| Selig S8038 | 29.2 | 6.25 |
| NACA 1412 | 29.6 | 6.25 |
| Eppler E226 | 30.4 | 6.25 |
| Mike Fox WASP | 31.1 | 6.25 |
| David Fraser DF101 | 31.5 | 6.25 |
| Selig S3016 | 31.8 | 6.25 |
| Selig/Donovan SD7080 | 33.6 | 6.25 |
| Selig/Donovan SD7043 | 38.6 | 6.25 |
| Selig/Donovan SD8040 | 32.6 | 6.5 |
| Selig/Donovan SD7037 | 34.5 | 6.5 |
| NACA 2410 | 35.3 | 6.5 |
| Eppler E64 | 41.5 | 6.5 |
| Selig S3014 | 32.5 | 6.75 |
| Eppler E224 | 33.7 | 6.75 |
| Eppler E212 | 38.3 | 6.75 |
| Eppler E58 | 46.1 | 6.75 |
| Eppler E62 | 48 | 6.75 |
| Eppler E180 | 35.5 | 7 |
| Eppler E222 | 36.5 | 7 |
| Selig/Donovan SD2083 | 37.4 | 7 |
| Selig/Ashok SA7025 | 38.1 | 7 |
| Selig S4053 | 38.2 | 7 |
| NACA 2412 | 32.5 | 7.25 |
| Eppler E49 | 37.5 | 7.25 |
| Eppler E186 | 29.9 | 7.5 |
| Selig/Donovan SD7032 | 31.9 | 7.5 |
| Eppler E178 | 36.7 | 7.5 |
| Eppler E211 | 37.9 | 7.5 |
| Eppler E221 | 29.4 | 7.75 |
| NACA 2411 | 34.3 | 7.75 |
| Selig S9037 | 34.7 | 7.75 |
| Eppler E176 | 37.7 | 7.75 |
| Eppler E66 | 40.2 | 7.75 |
| Eppler E168 | 22.6 | 8 |
| Eppler E374 | 34.1 | 8 |
| Selig/Ashok SA7026 | 38.4 | 8 |
| Selig S4062 | 38.7 | 8 |
| Eppler E216 | 40.7 | 8 |
| Selig S3021 | 32.2 | 8.25 |
| Selig S3024 | 34.4 | 8.25 |
| Eppler E174 | 38.1 | 8.25 |
| Drela DAE51 | 39.3 | 8.25 |
| Selig S5020 | 25.3 | 8.5 |
| Drela Apex 16 | 34 | 8.5 |

| | | |
|---|---|---|
| Selig S4061 | 35.7 | 8.5 |
| Selig/Donovan SD6080 | 36.1 | 8.5 |
| Selig S5010 | 24.1 | 8.75 |
| NACA 2414 | 29.2 | 8.75 |
| Eppler E220 | 31.8 | 8.75 |
| Selig S3025 | 33.9 | 8.75 |
| NACA 2415 | 27.4 | 9 |
| Selig S4320 | 32.3 | 9 |
| Eppler E193 | 35.8 | 9 |
| Eppler E392 | 37.2 | 9 |
| Eppler E214 | 37.9 | 9 |
| Eppler E205 | 32.6 | 9.25 |
| Eppler E201 | 34 | 9.25 |
| Selig S4180 | 28.3 | 9.5 |
| Selig S2091 | 28.9 | 9.5 |
| Eppler E195 | 32.8 | 9.5 |
| Eppler E207 | 30.9 | 9.75 |
| Selig S7055 | 31.1 | 9.75 |
| Eppler E203 | 32 | 9.75 |
| NACA 4412 | 33.4 | 9.75 |
| Selig/Donovan SD7034 | 27.8 | 10 |
| Eppler E209 | 29.4 | 10 |
| Selig S4310 | 28.2 | 10.5 |
| Eppler E197 | 29 | 10.5 |
| NACA 6409 | 27.1 | 10.75 |

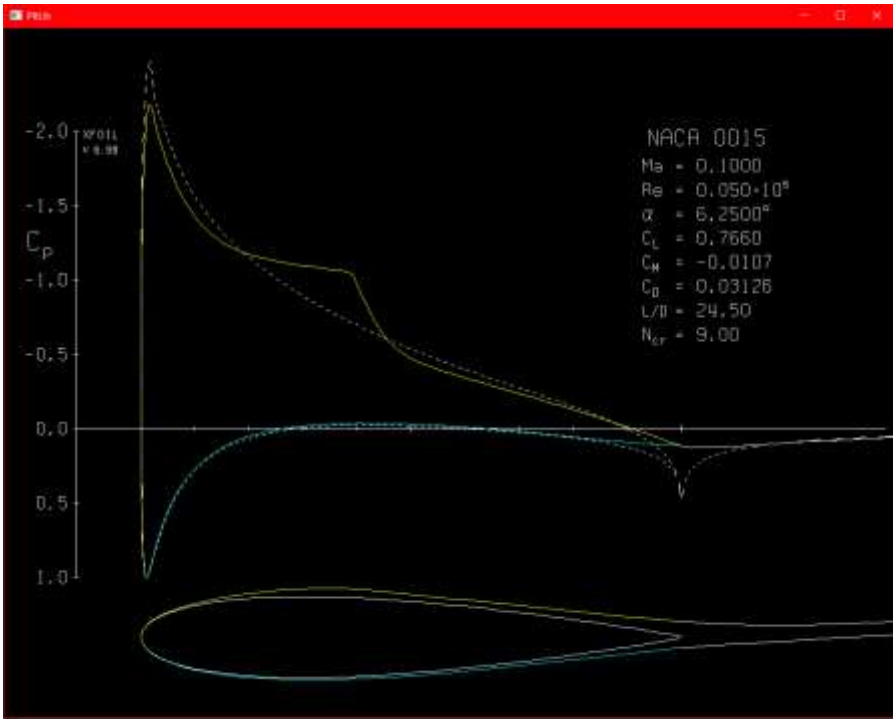
**Appendix A: A Table showing the Methodology of Airfoil Selection.**



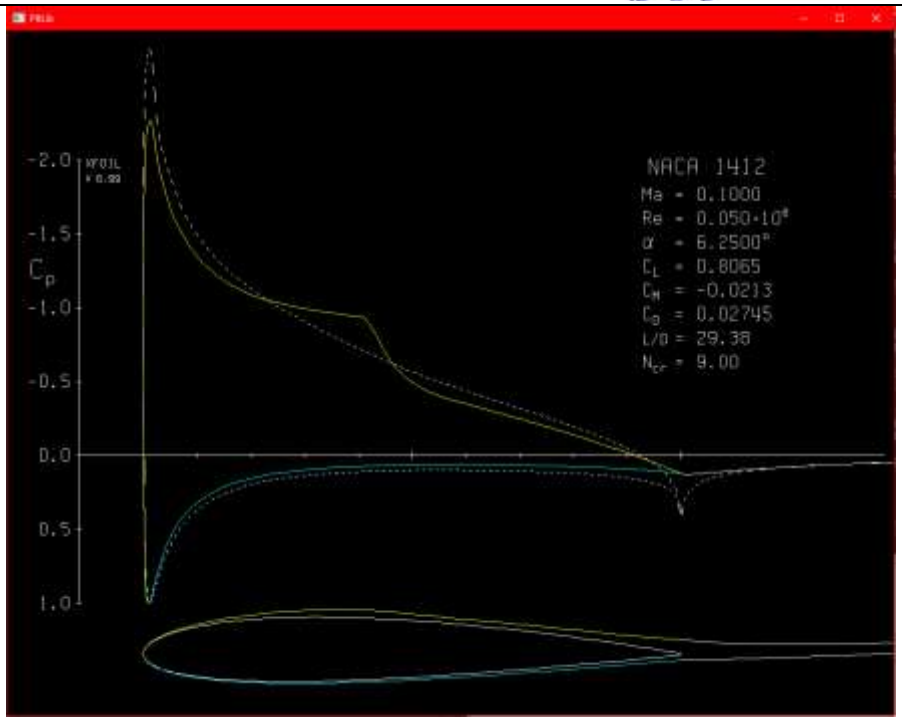

**Appendix B1: Eppler E171 Pressure Distribution Simulation.**



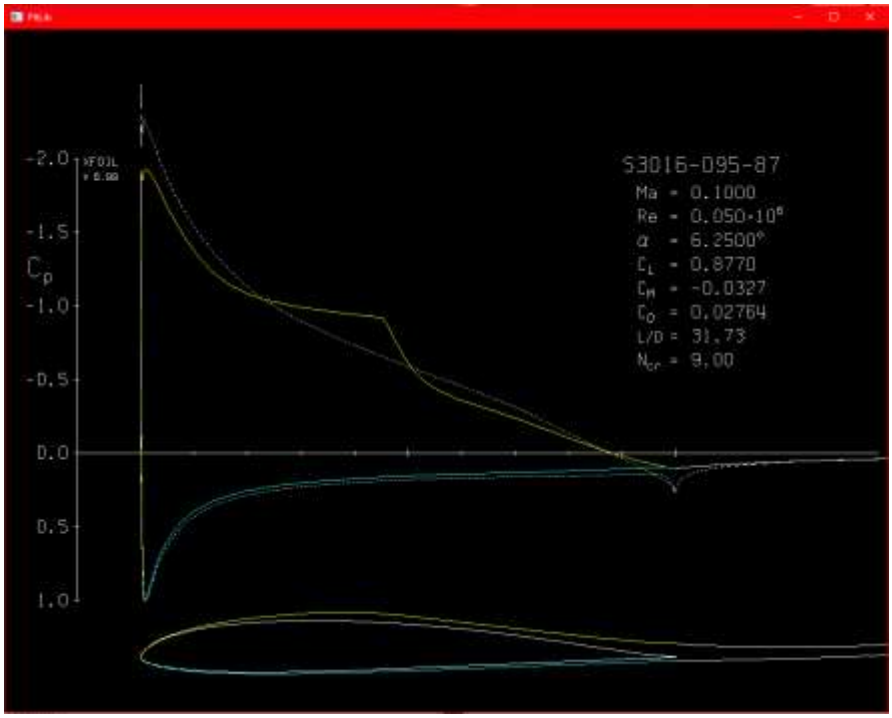

**Appendix B2: Eppler E226 Pressure Distribution Simulation.**



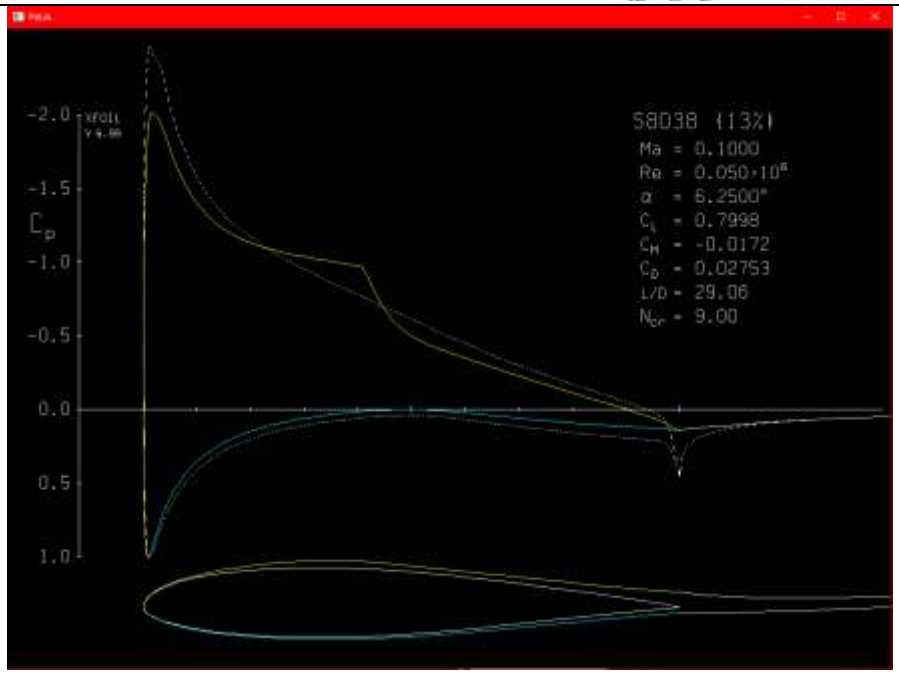

**Appendix B3: NACA 0015 Pressure Distribution Simulation.**



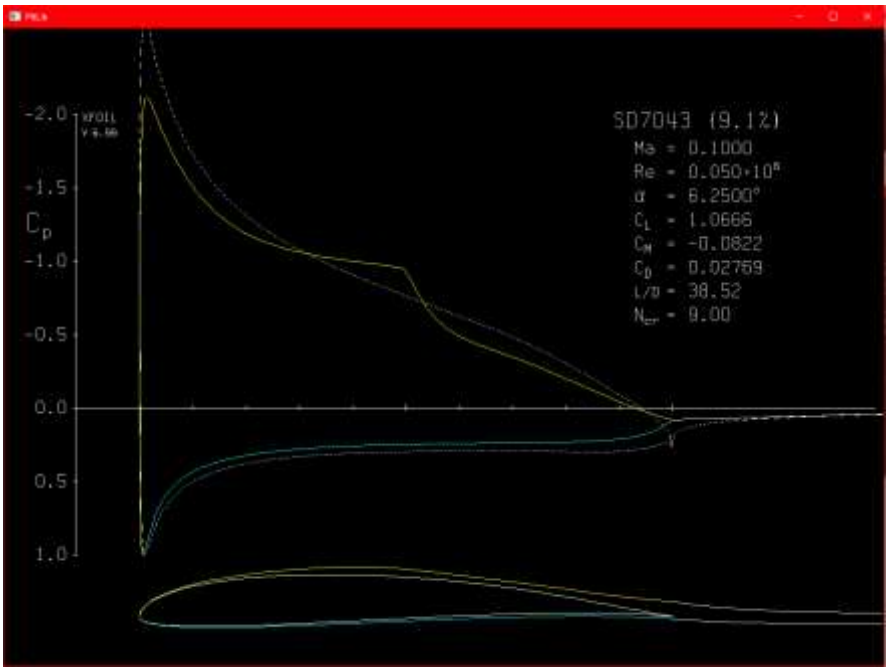

**Appendix B4: NACA 1412 Pressure Distribution Simulation.**



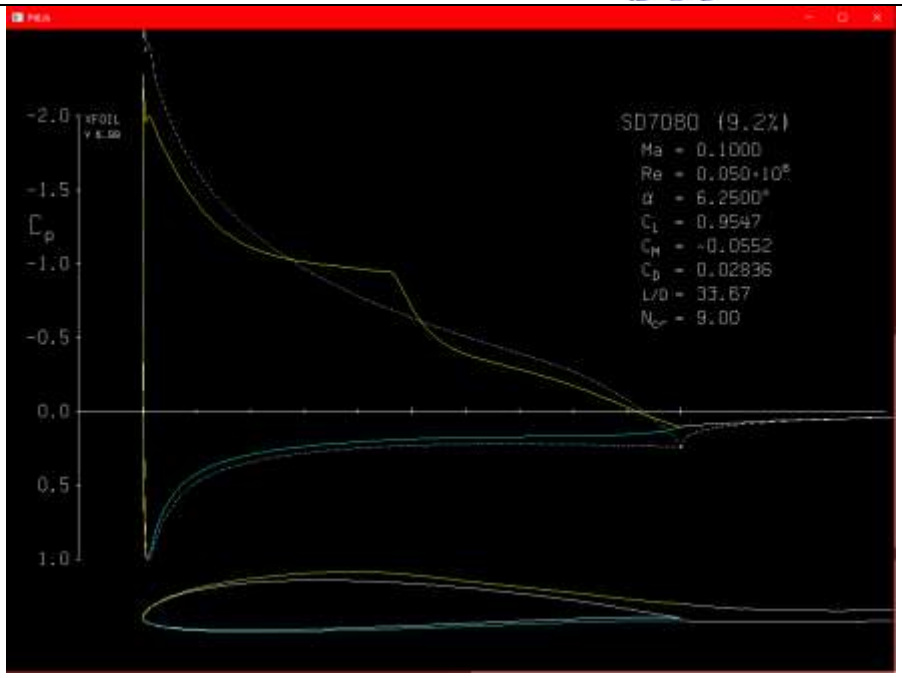

**Appendix B5: Selig S3016 Pressure Distribution Simulation.**



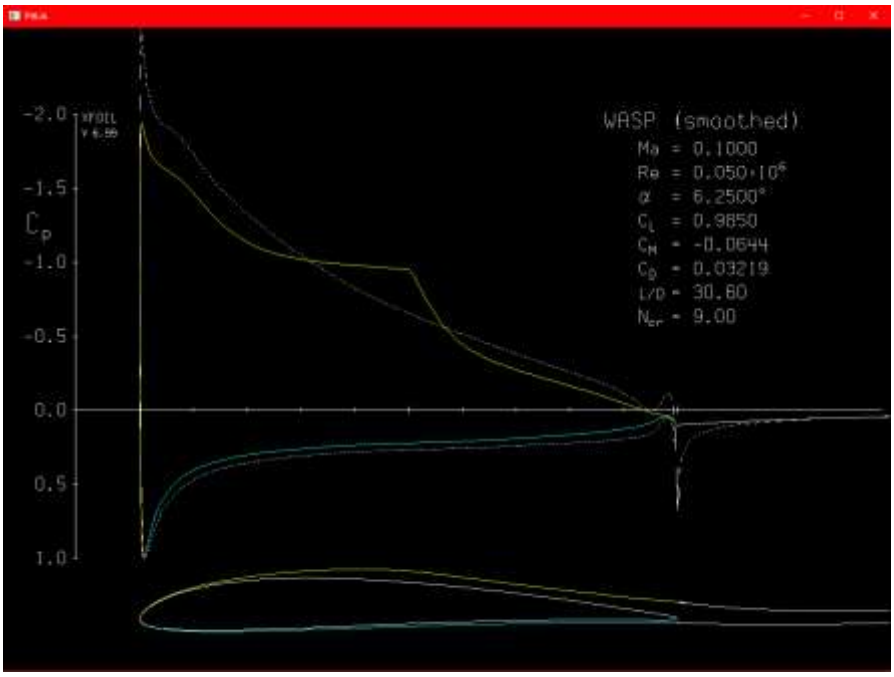

**Appendix B6: Selig S8038 Pressure Distribution Simulation.**



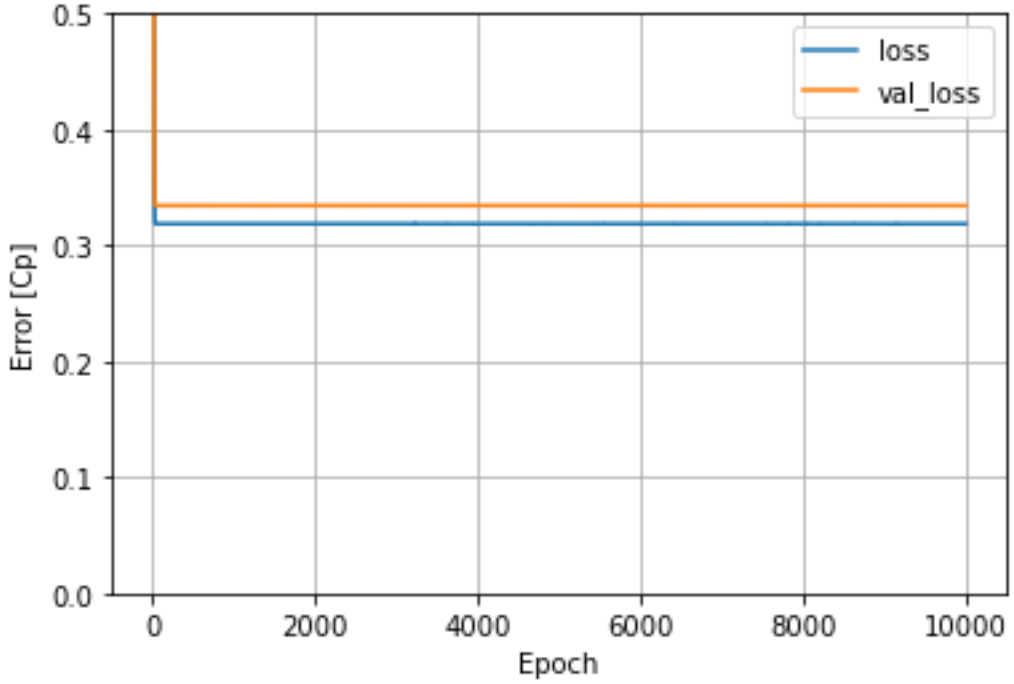

**Appendix B7: Selig/Donovan SD7043 Pressure Distribution Simulation.**



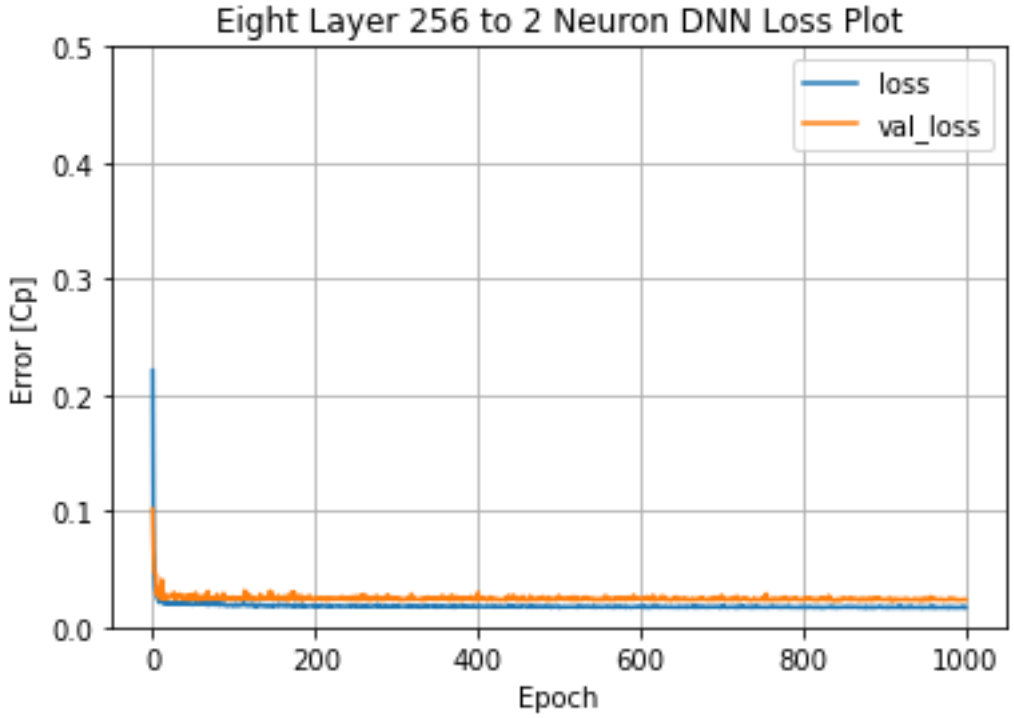

**Appendix B8: Selig/Donovan SD7080 Pressure Distribution Simulation.**



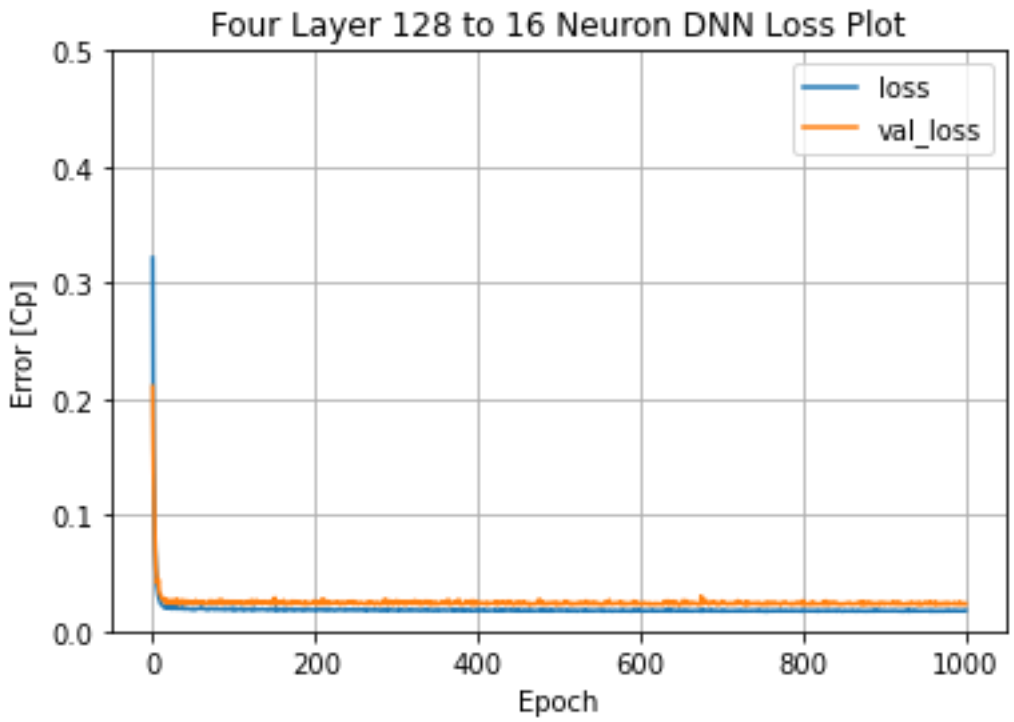

**Appendix B9: Mike Fox WASP Pressure Distribution Simulation.**



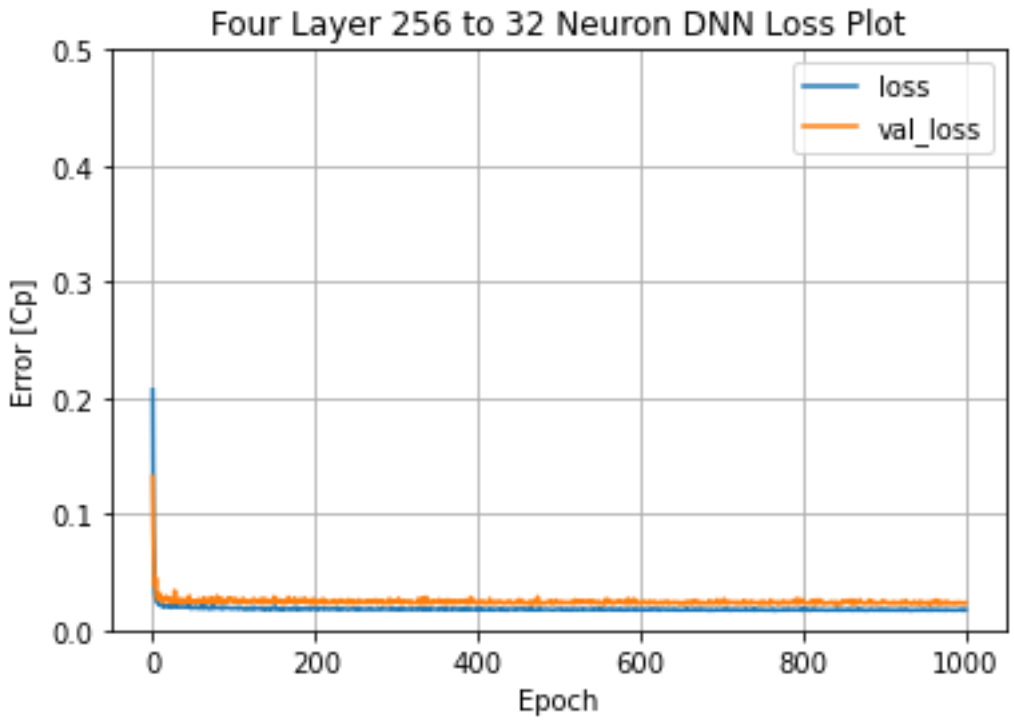

**Appendix C1: Linear Neural Network Loss Plot.**



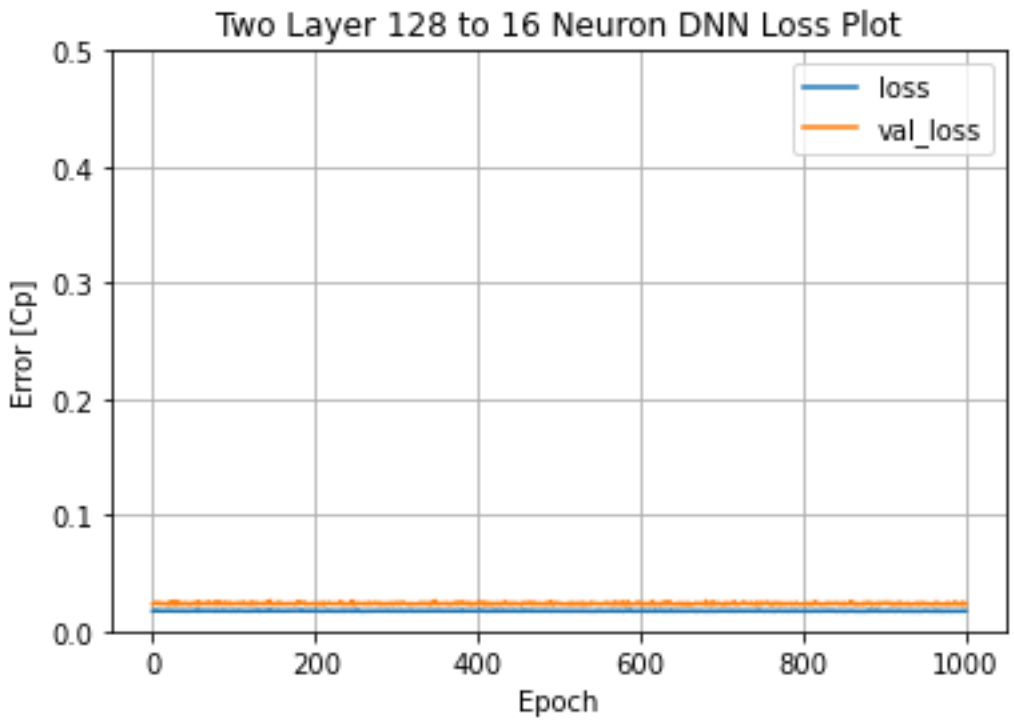

**Appendix C2: Eight Layer 256 to 2 Neuron LeakyReLU DNN Loss Plot.**



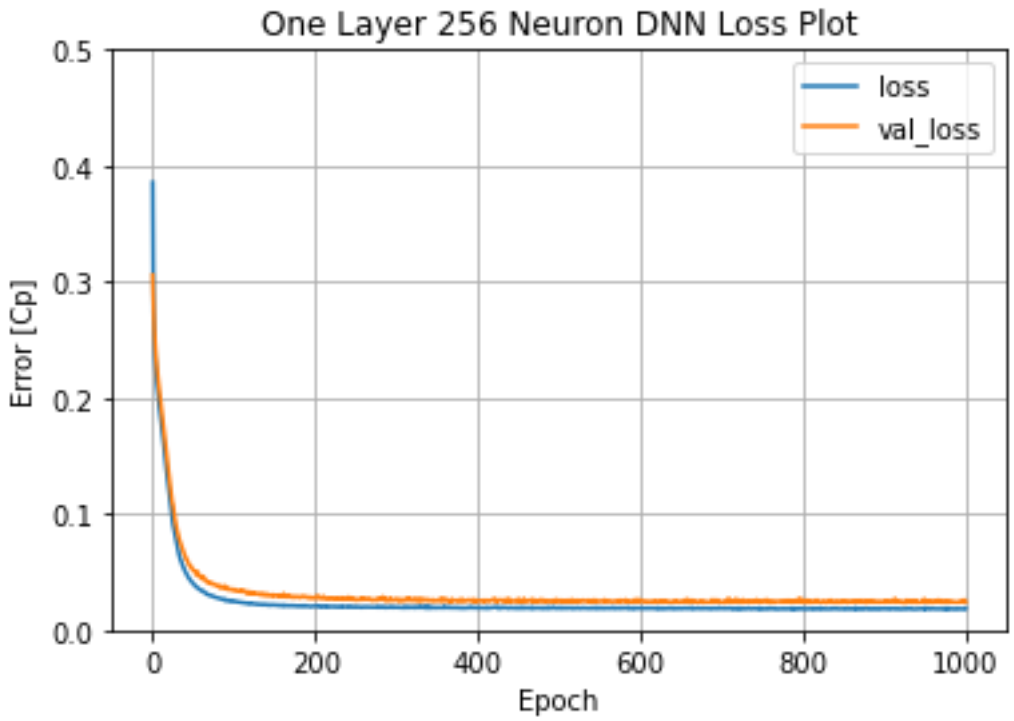

**Appendix C3: Four Layer 128 to 16 Neuron LeakyReLU DNN Loss Plot.**



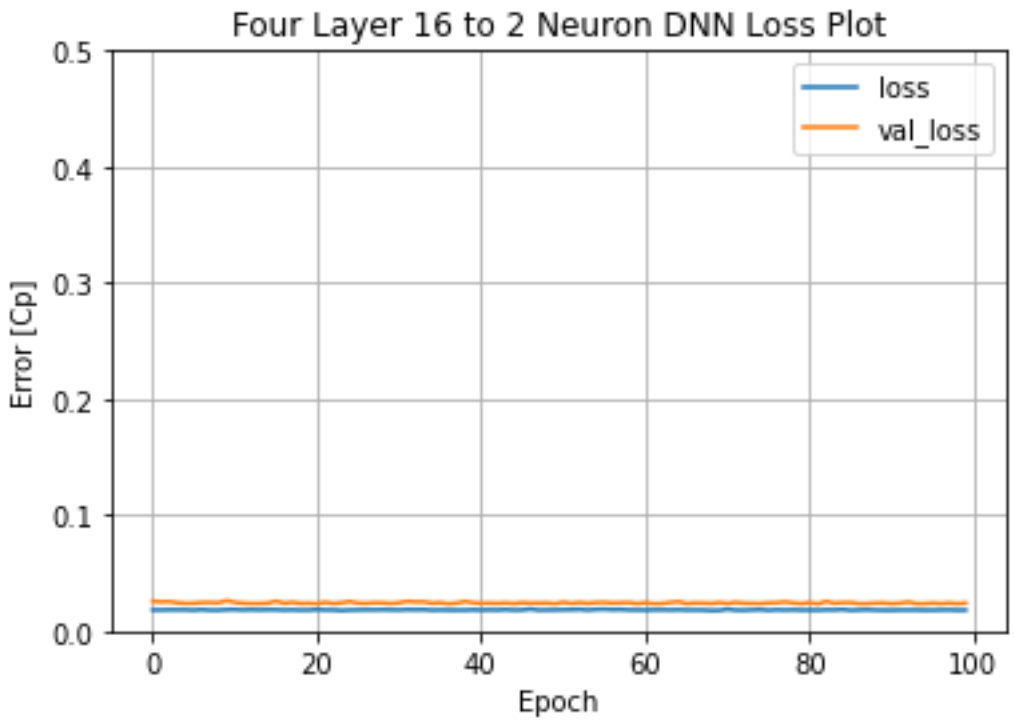

**Appendix C4: Four Layer 256 to 32 Neuron LeakyReLU DNN Loss Plot.**



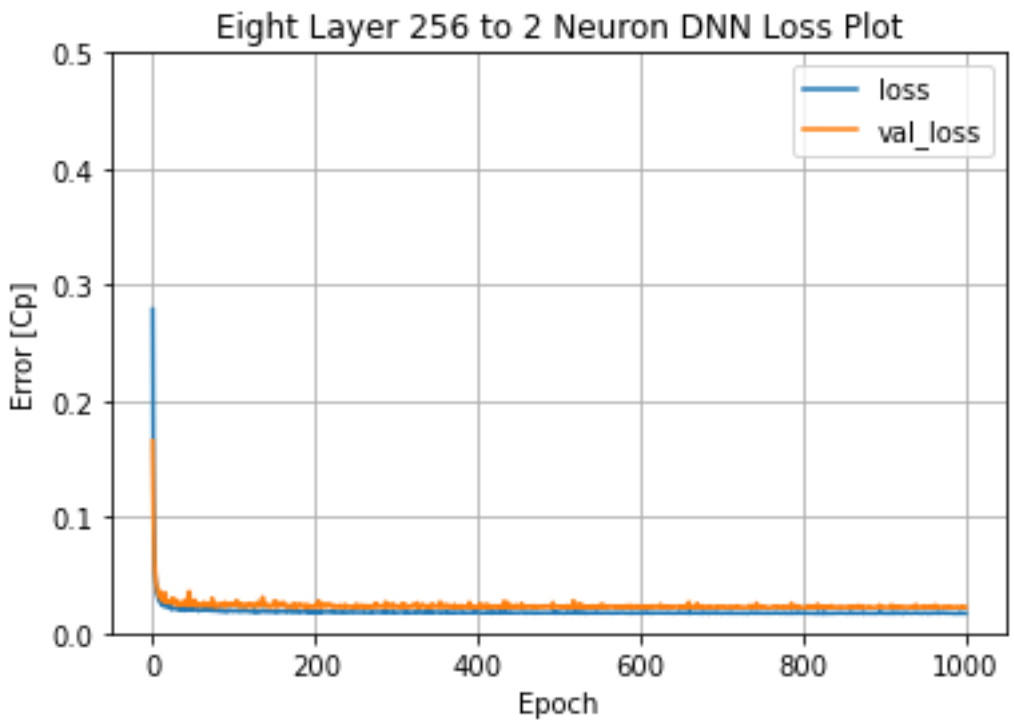

**Appendix C5: Two Layer 128 to 16 Neuron LeakyReLU DNN Loss Plot.**



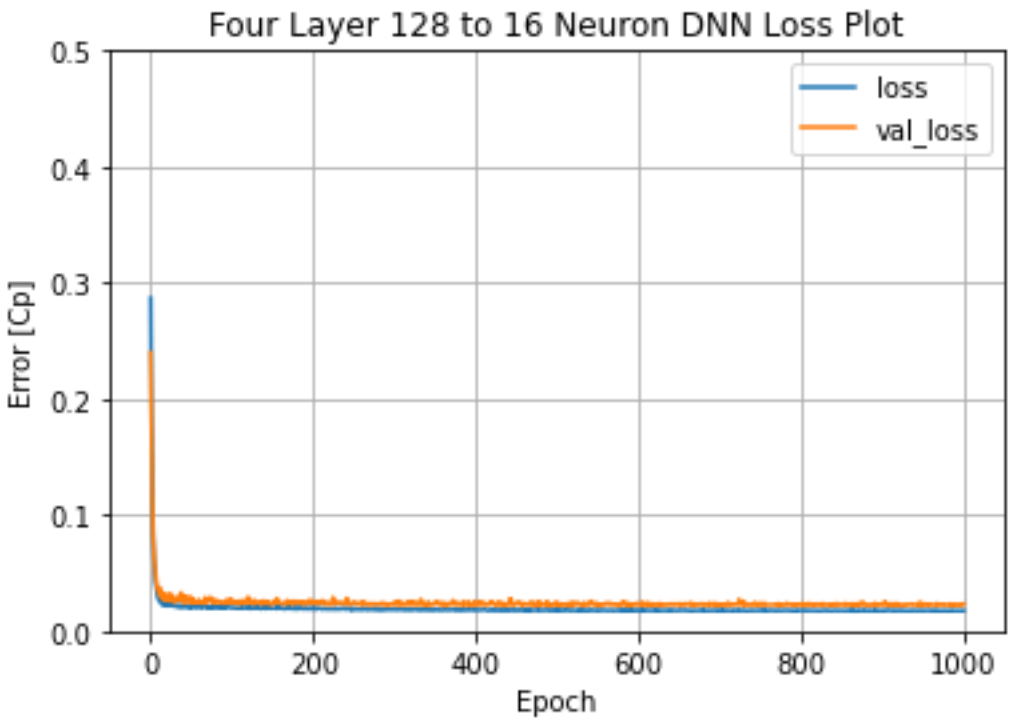

**Appendix C6: One Layer 256 Neuron LeakyReLU DNN Loss Plot.**



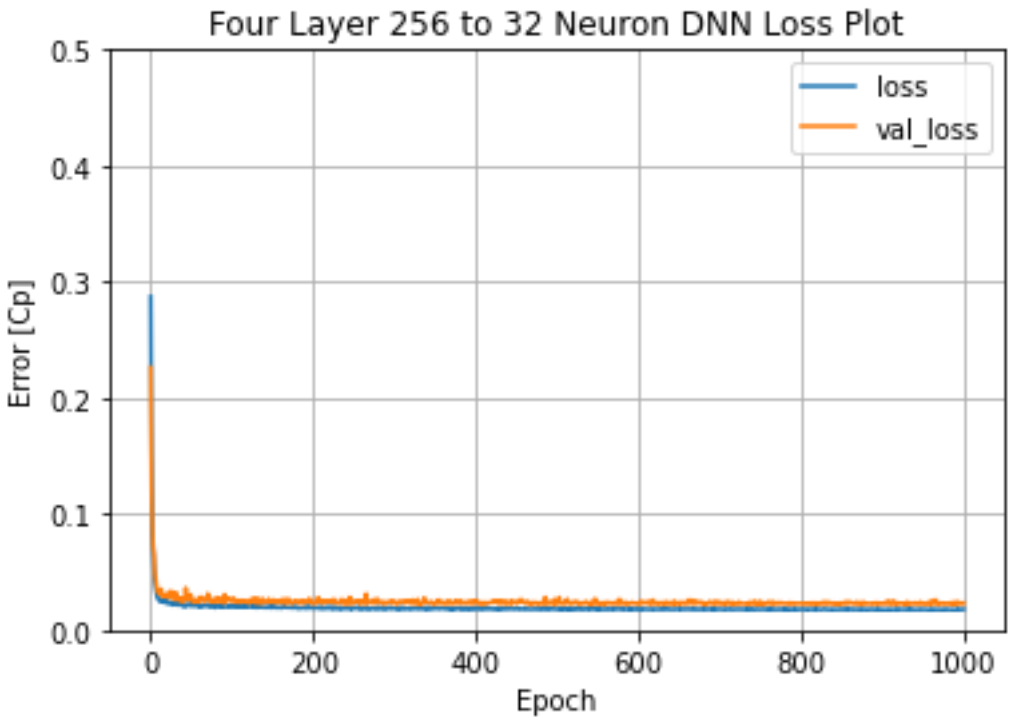

**Appendix C7: Four Layer 16 to 2 Neuron LeakyReLU DNN Loss Plot.**



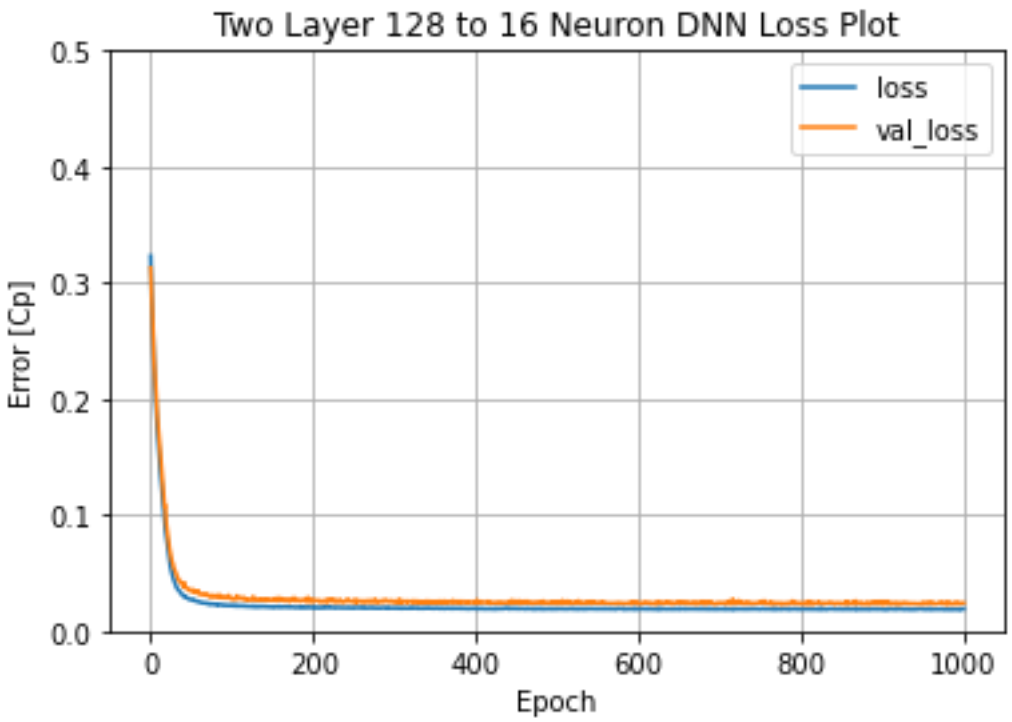

**Appendix C8: Eight Layer 256 to 2 Neuron ELU DNN Loss Plot.**



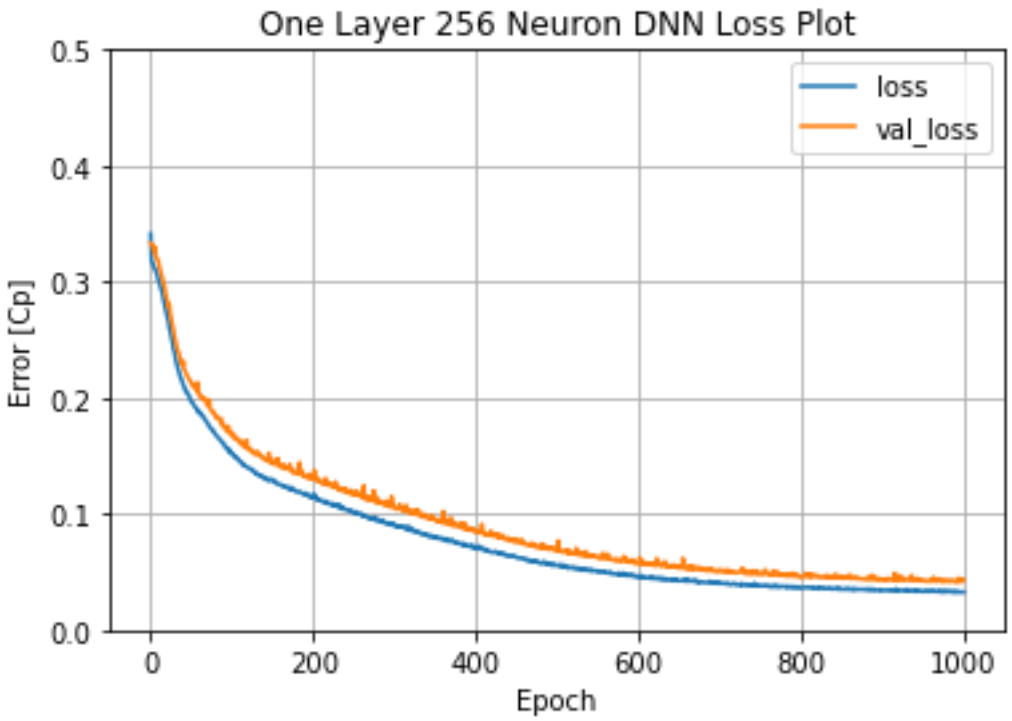

**Appendix C9: Four Layer 128 to 16 Neuron ELU DNN Loss Plot.**



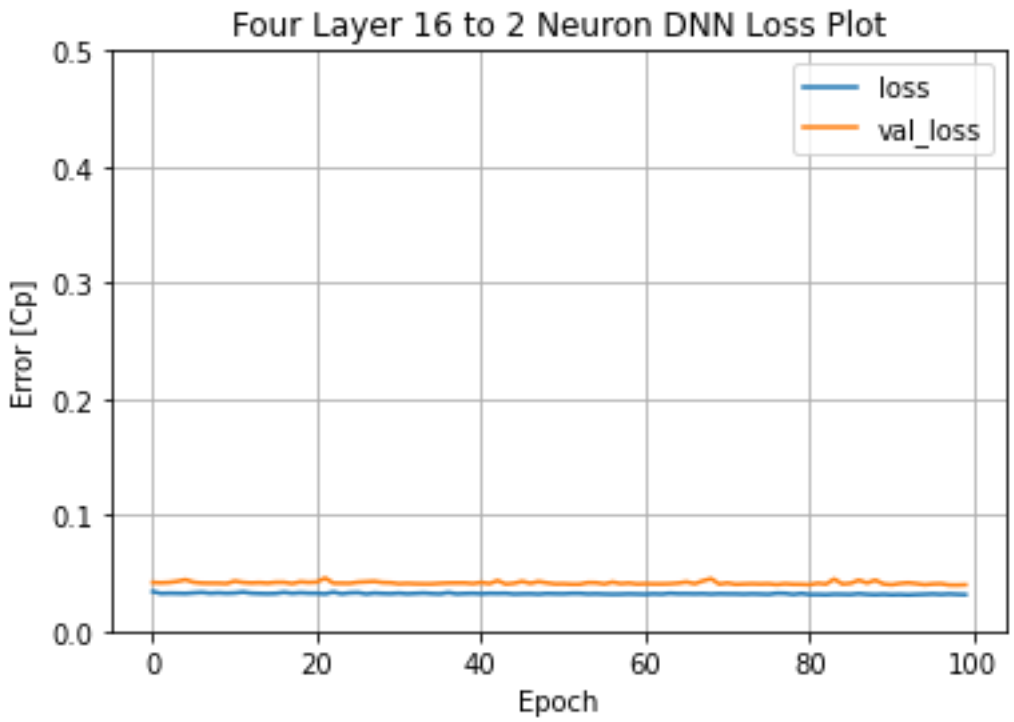

**Appendix C10: Four Layer 256 to 32 Neuron ELU DNN Loss Plot.**



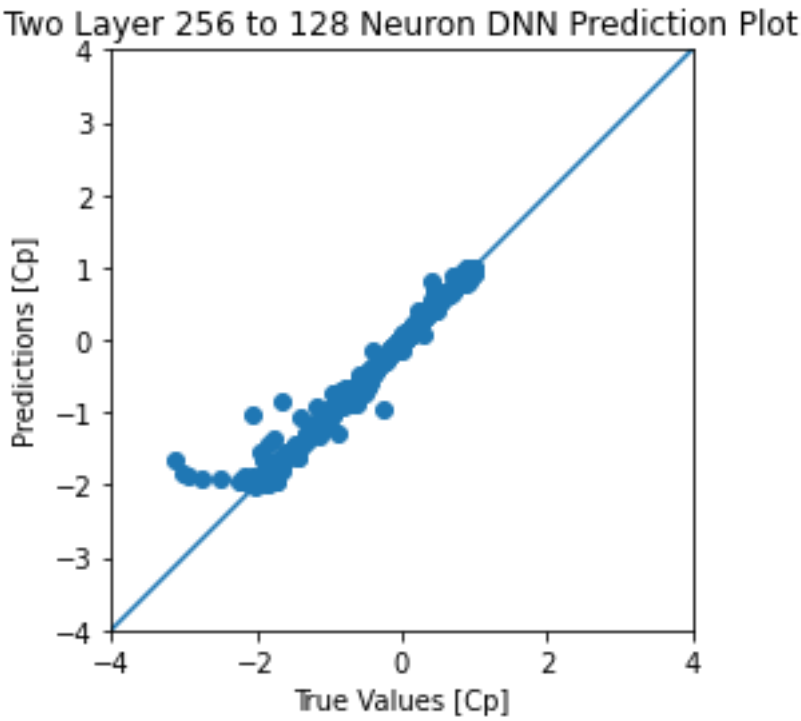

**Appendix C11: Two Layer 128 to 16 Neuron ELU DNN Loss Plot.**



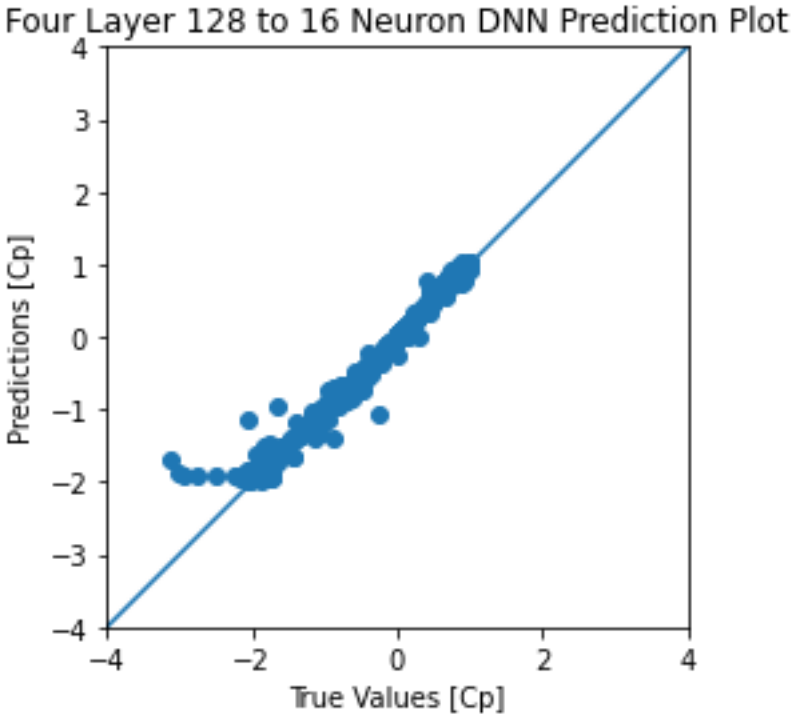

**Appendix C12: One Layer 256 Neuron ELU DNN Loss Plot.**



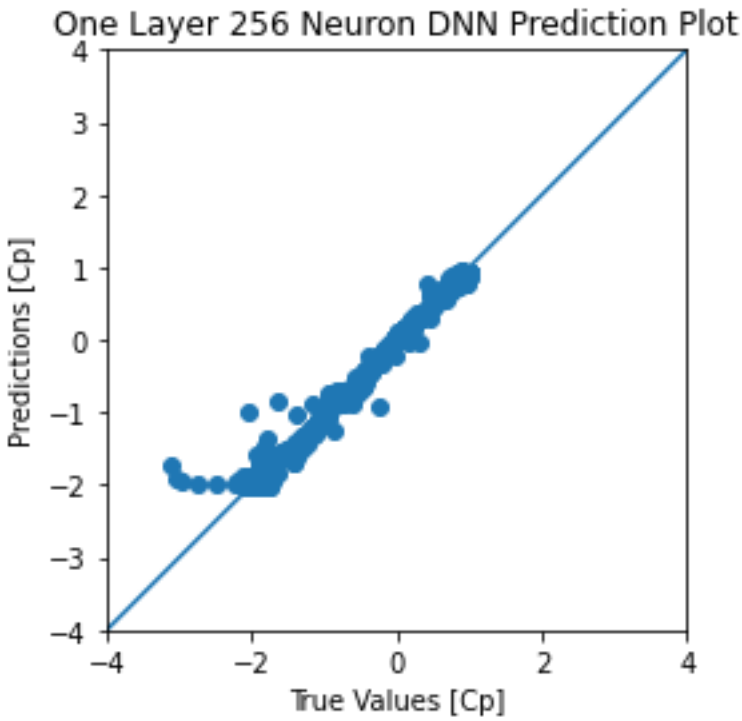

**Appendix C13: Four Layer 16 to 2 Neuron ELU DNN Loss Plot.**



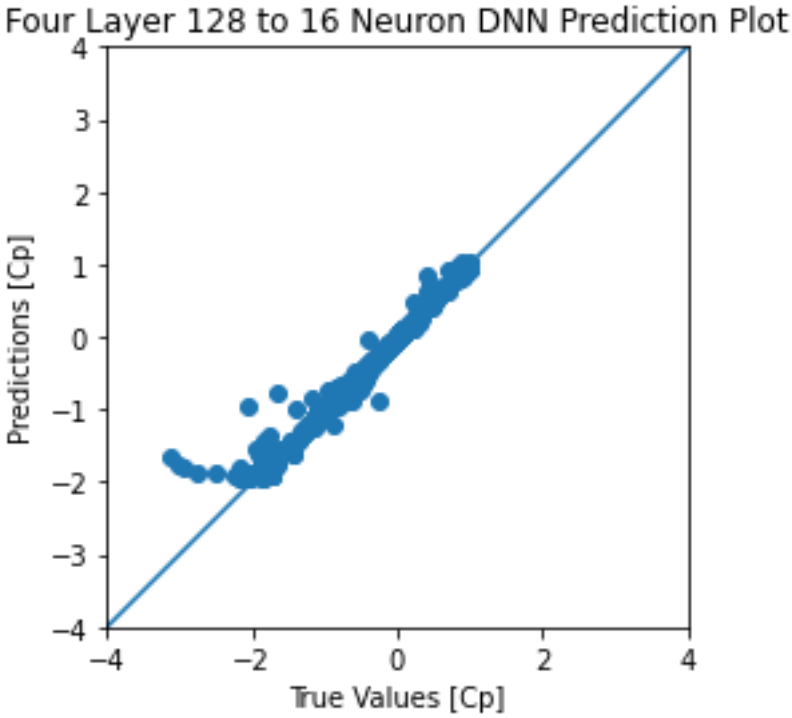

**Appendix D1: Two Layer 256 to 128 Neuron ReLU DNN Prediction Plot.**



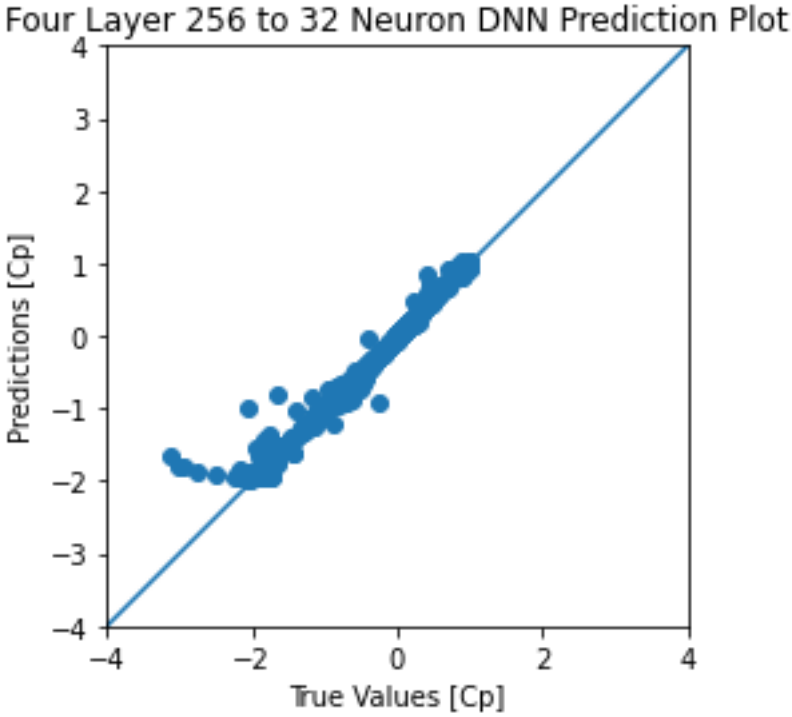

**Appendix D2: Four Layer 128 to 16 Neuron ReLU DNN Prediction Plot.**



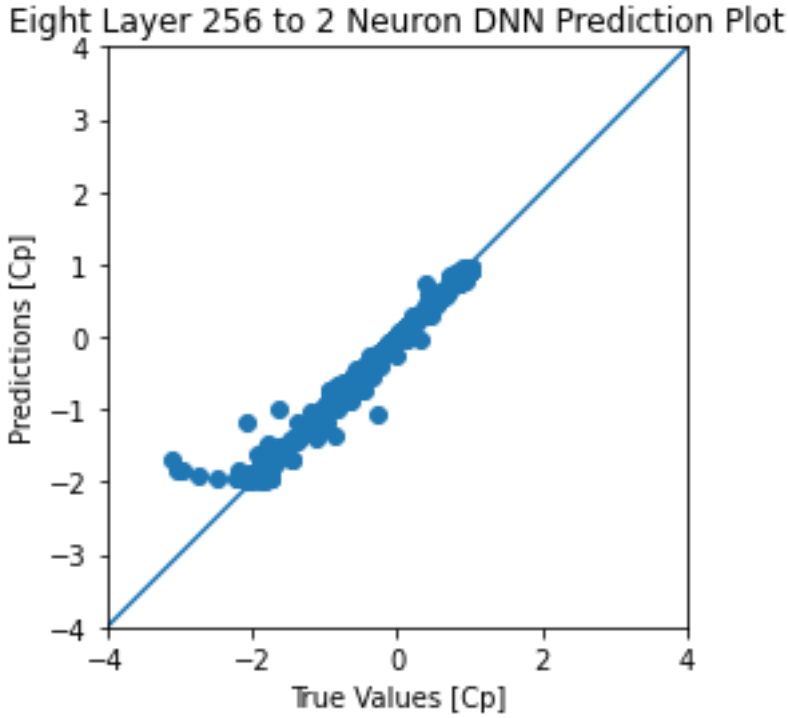

**Appendix D3: One Layer 256 Neuron ReLU DNN Prediction Plot.**



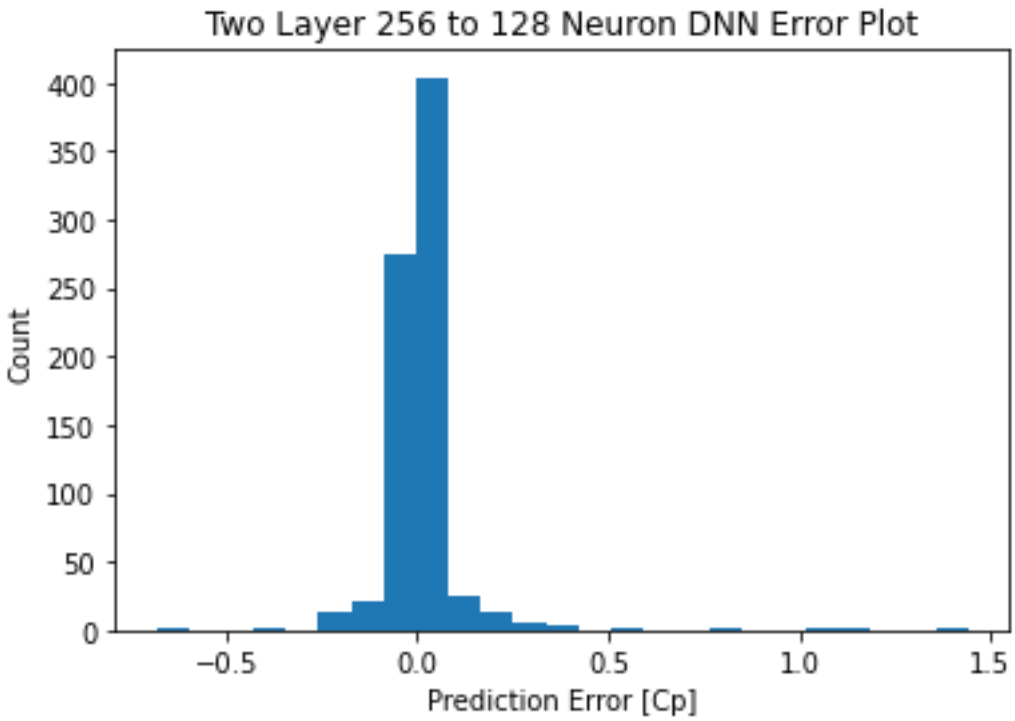

**Appendix D4: Two Layer 128 to 16 Neuron ELU DNN Prediction Plot.**



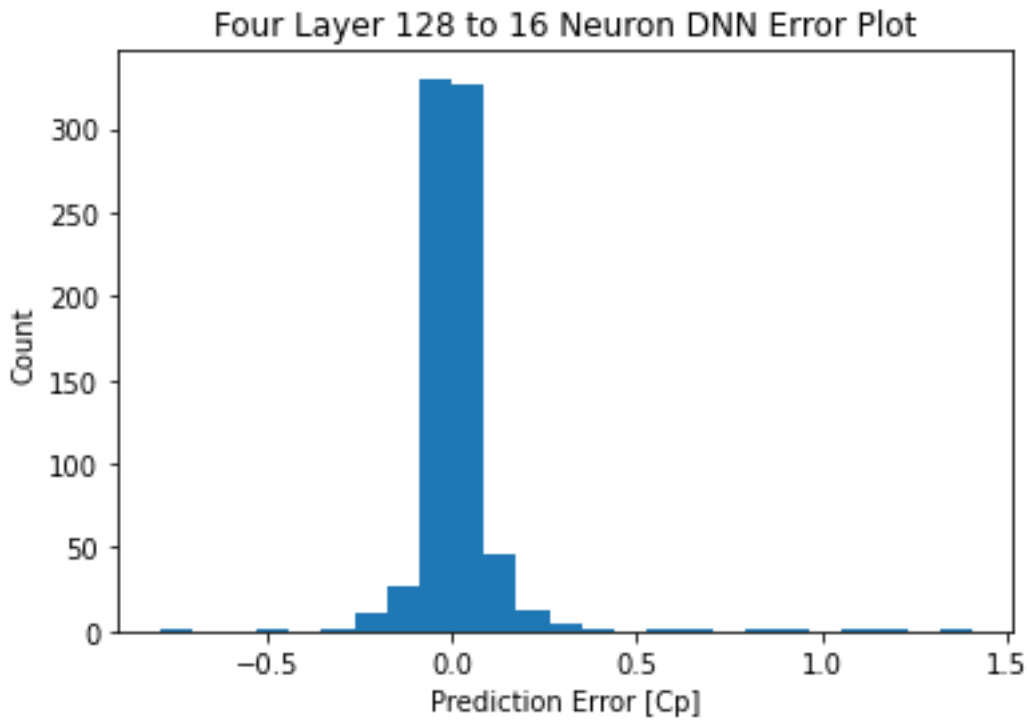

**Appendix D5: Four Layer 256 to 32 Neuron ELU DNN Prediction Plot.**



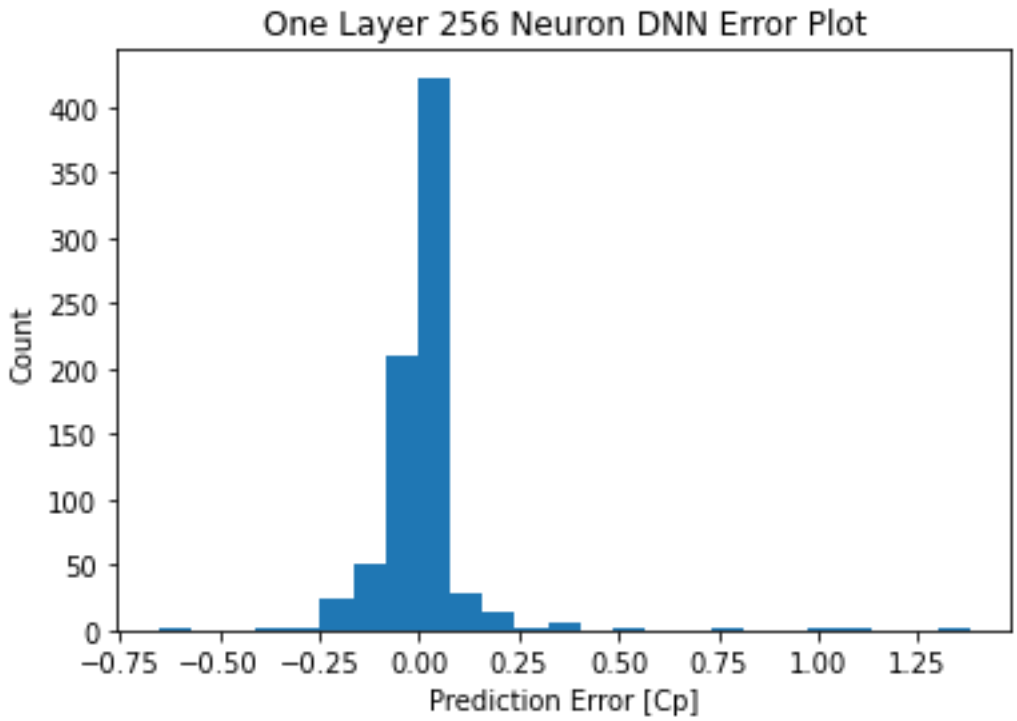

**Appendix D6: Eight Layer 256 to 2 Neuron DNN Prediction Plot.**



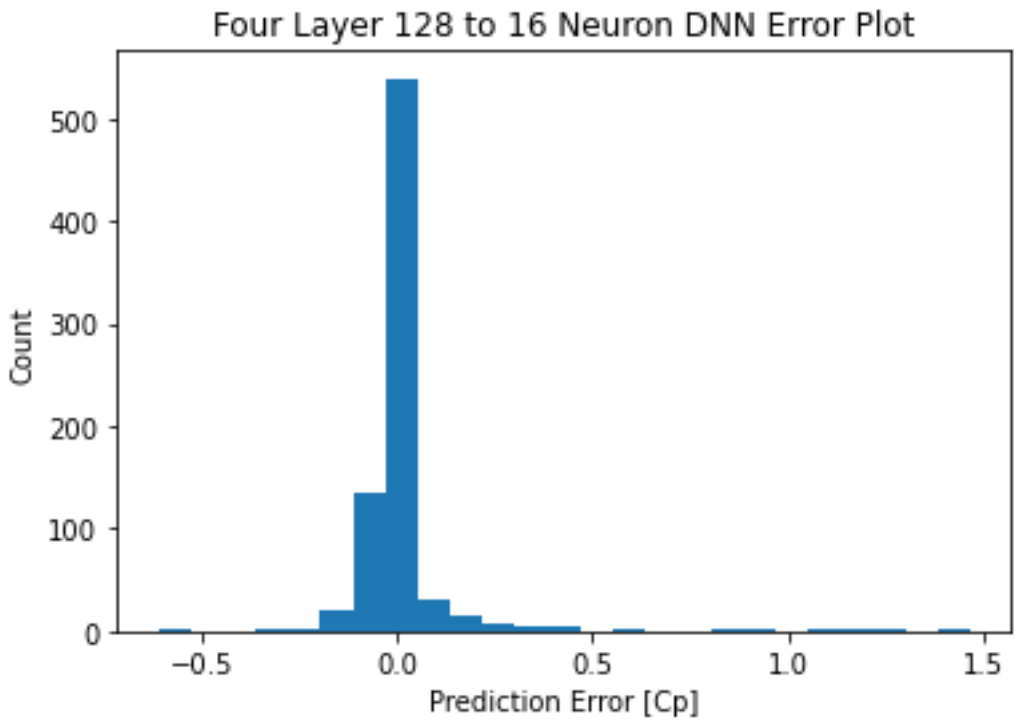

**Appendix E1: Two Layer 256 to 128 Neuron ReLU DNN Error Plot.**



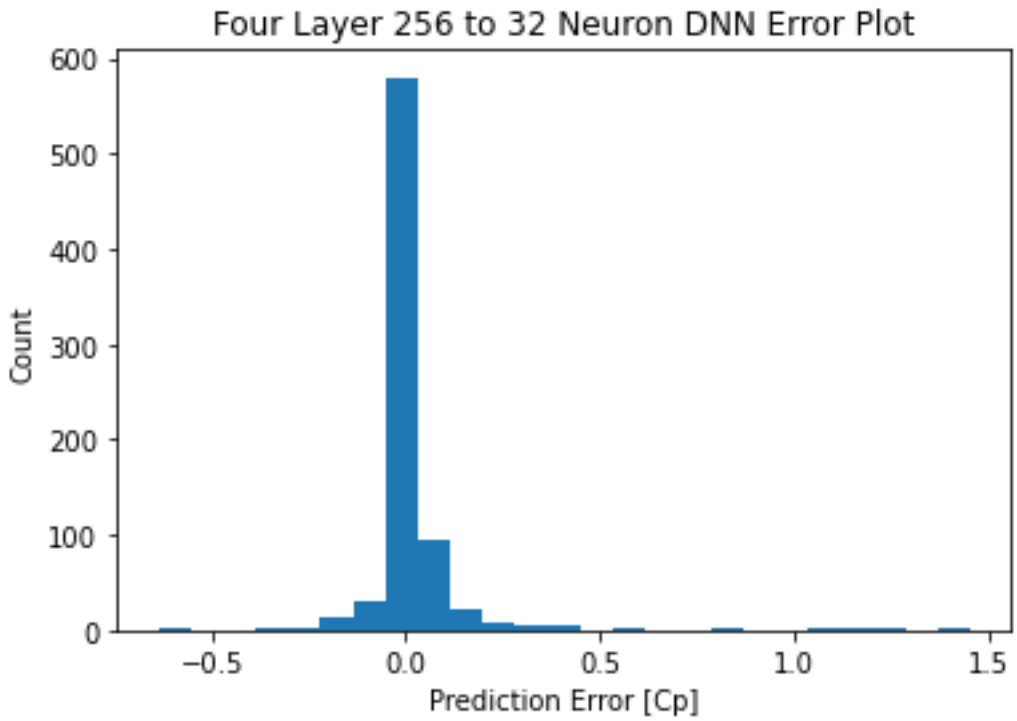

**Appendix E2: Four Layer 128 to 16 Neuron ReLU DNN Error Plot.**



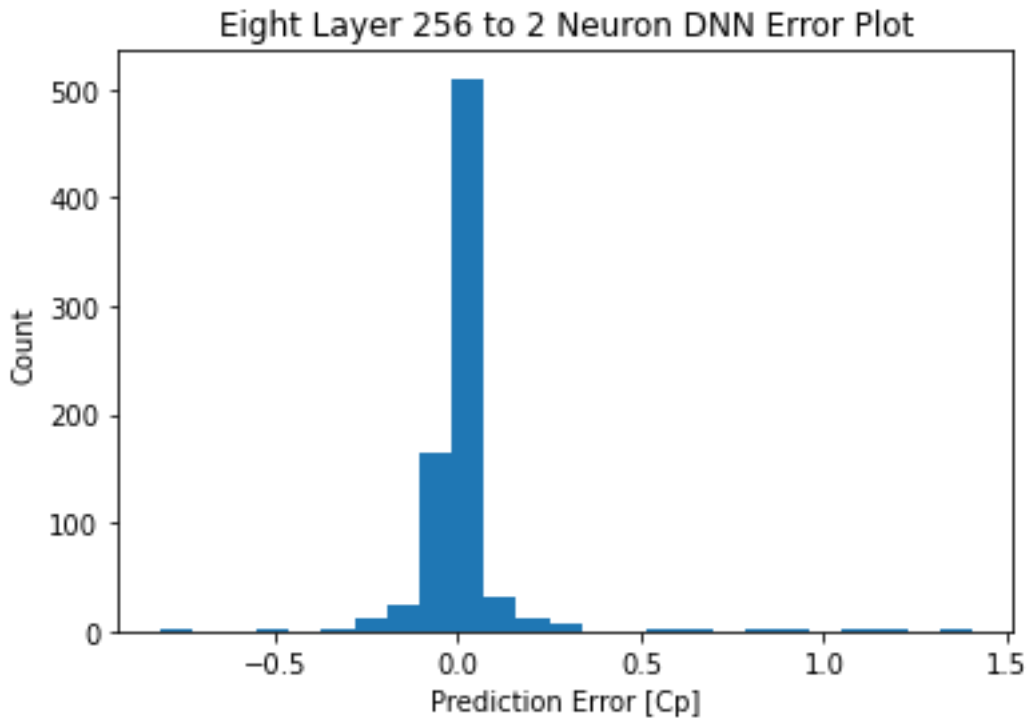

**Appendix E3: One Layer 256 Neuron ReLU DNN Error Plot.**




**Appendix E4: Two Layer 128 to 16 Neuron ELU DNN Error Plot.**




**Appendix E5: Four Layer 256 to 32 Neuron ELU DNN Error Plot.**




**Appendix E6: Eight Layer 256 to 2 Neuron DNN Error Plot.**

# 12 Final Scripts

## 12.1 Preprocessing

```
#!/usr/bin/env python
# coding: utf-8

# In[1]:



import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from pandas import DataFrame
get_ipython().run_line_magic('matplotlib', 'inline')



# In[2]:
```

```
# Read the Pressure Distribution file from XFoil, reading the text
# file, as a fixed width delimited file, skipping the first two rows.
widths = [1,9,9,9]
df = pd.read_fwf('WASP.cp', skiprows = ([0,1]), widths = widths)
print(df)


# In[3]:


# Remove the first column as it is unnecessary.
df = df.drop(columns = '#')
print(df)


# In[9]:


# Inspect the data for null values.
df.isna().sum()


# In[4]:


df.dtypes


# In[5]:


# Save the newly converted dataframe, for future use
df.to_pickle('WASP.pickle')
```

```
# In[6]:


# Delete the initial dataframe.
del df


# In[7]:


# Read the saved dataframe, to verify the data.
df = pd.read_pickle('WASP.pickle')
print(df)


# In[8]:


# Plotting the airfoil as a scatter diagram, showing the pressure coefficient as a colormap, to verify the preprocessing.
ax = df.plot.scatter(x = "x", y = "y", c = "Cp", colormap='jet', figsize = (15,5))

# adding vertical line in data co-ordinates
plt.axvline(0, c='black', ls='--')

# adding horizontal line in data co-ordinates
plt.axhline(0, c='black', ls='--')
plt.title("WASP Airfoil plot showing Pressure Coefficient")
plt.savefig('WASP.png')
plt.show()
```

## 12.2 Dataframe

```
#!/usr/bin/env python
# coding: utf-8

# In[3]:


import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from pandas import DataFrame


# In[30]:


df1 = pd.read_pickle('DF101.pickle')
df2 = pd.read_pickle('E171.pickle')
df3 = pd.read_pickle('E226.pickle')
df4 = pd.read_pickle('NACA 0015.pickle')
df5 = pd.read_pickle('NACA 1412.pickle')
df6 = pd.read_pickle('S3016.pickle')
df7 = pd.read_pickle('S8038.pickle')
df8 = pd.read_pickle('SD7043.pickle')
df9 = pd.read_pickle('SD7080.pickle')
df10 = pd.read_pickle('WASP.pickle')


# In[47]:


# Append all dataframes to the initial dataframe.
```

```
orig_df = df1.append((df1,df2, df3, df4, df5, df6, df7, df8, df9, df10), ignore_index =
True)
print("\nAppended DataFrame:")
print (orig_df)
orig_df.describe().transpose()



# In[ ]:





# In[48]:



# shuffle the DataFrame rows
df = df.sample(frac = 1)

# print the shuffled DataFrame
print("\nShuffled DataFrame:")
print(df)



# In[49]:



# Save the newly converted dataframe, for future use
df.to_pickle('Appended Data.pickle')



# In[50]:



ax = df.plot.scatter(x = "x", y = "y", c = "Cp", colormap='jet', figsize = (15,5))
```

```
plt.title("Appended Airfoil plot showing Pressure Coefficients")
plt.savefig('Appended.png')
plt.show()


# In[ ]:





# In[ ]:
```

## 12.3 LeakyReLU Model

```
#!/usr/bin/env python
# coding: utf-8

# In[1]:


import pandas as pd

# Import the following libraries, for slicing.
from sklearn.model_selection import train_test_split

# To manipulate the data set as a data frame, import pandas.
import pandas as pd

# For management and plotting histograms, import MatplotLib.
import matplotlib.pyplot as plt

# Import the following libraries, for slicing, cross validating and tuning, respectively.
```

```
from sklearn.model_selection import train_test_split, cross_val_score, RandomizedSearchCV

# Import the classification report, accuracy score and confusion matrix functions, to evaluate the model.
from sklearn.metrics import accuracy_score

# Import the pprint function, to print python data structures as inputs.
from pprint import pprint

# To define the arrays for variables, import NumPy.
import numpy as np

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns


# Make numpy printouts easier to read.
np.set_printoptions(precision=3, suppress=True)


# In[2]:


import tensorflow as tf

from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.layers.experimental import preprocessing

print(tf.__version__)
```

```
# In[3]:


#Import the data.
df = pd.read_pickle('Appended data.pickle')


# In[4]:


# Visualize the distibution of the data.
g = sns.pairplot((df), diag_kind='kde')
g.fig.suptitle("Distribution of Airfoil Data", y=1.08)
plt.show()


# In[5]:


# Use the corr function to calculate the correlations within the dataset, and define them for use in a heatmap.
corrdata = df.corr()

# Plot a heatmap to show the correlation of the data within the dataset.
plt.title("Correlations within Airfoil Data")
sns.heatmap(corrdata)
plt.show()


# In[6]:


# Splitting the data nto training and test sets.
train_dataset = df.sample(frac=0.8, random_state=0)
test_dataset = df.drop(train_dataset.index)
```

```
# In[7]:


# Inspect the statistics of the training data.
print (train_dataset.describe().transpose())
# Previewing the training data.
print(train_dataset)
# Visualize the distibution of the data.
sns.pairplot(train_dataset[['Cp', 'x', 'y']], diag_kind='kde')
# Inspect the statistics of the test data.
print (test_dataset.describe().transpose())
# Add titles using /n
print(test_dataset)
# Visualize the distibution of the data.
sns.pairplot(test_dataset[['Cp', 'x', 'y']], diag_kind='kde')


# In[8]:


# Visualize the distibution of the data.
sns.pairplot(train_dataset[['Cp', 'x', 'y']], diag_kind='kde')


# In[9]:


# Visualize the distibution of the data.
sns.pairplot(test_dataset[['Cp', 'x', 'y']], diag_kind='kde')


# In[10]:
```

```
# Splitting the feature set from the label set.
train_features = train_dataset.copy()
test_features = test_dataset.copy()

train_labels = train_features.pop('Cp')
test_labels = test_features.pop('Cp')


# In[11]:


# Normalization of the data. Increases model stability.
# Inspect the ranges of each feature
train_dataset.describe().transpose()[['mean', 'std']]


# In[12]:


# Create the integrated normalization layer.
normalizer = preprocessing.Normalization()


# In[13]:


# Adapt the newly created layer to the dataset.
normalizer.adapt(np.array(train_features))


# In[14]:


# The adaptation calculates and stores the variance and mean of the dataset.
```

```
print(normalizer.mean.numpy())


# In[15]:


# Calling the layer to return the normalized features.
first = np.array(train_features[:1])

with np.printoptions(precision=2, suppress=True):
  print('First example:', first)
  print()
  print('Normalized:', normalizer(first).numpy())


# In[16]:


# Calling a batch of inputs to verify the model is producing the expected data.
#print(linear_model.predict(train_features[:10]))

# Calling the model to initialize its weight matrices and verifying the shape of the kernel.
#print(linear_model.layers[1].kernel)


# In[30]:


#LINEAR MODEL
# Create the linear regression model. (Model 1)
linear_model = tf.keras.Sequential([normalizer,layers.Dense(units=1)])

# Configuration of the training model by specifying the loss and optimizer functions.
linear_model.compile(
```

```
    optimizer=tf.optimizers.Adamax(learning_rate=0.001),
    loss='mse')

# Execution of the training of the linear model.
get_ipython().run_line_magic('time', 'history = linear_model.fit(train_features, train_labels, epochs=10000, verbose=0, validation_split = 0.2)')

# Visualization of the progress of training the model.
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

def plot_loss(history):
  plt.plot(history.history['loss'], label='loss')
  plt.plot(history.history['val_loss'], label='val_loss')
  plt.ylim([0, 0.5])
  plt.xlabel('Epoch')
  plt.ylabel('Error [Cp]')
  plt.legend()
  plt.grid(True)

# Define the test results dataframe and save the initial model's mean squared error.
test_results = {}
test_results['linear_model'] = linear_model.evaluate(test_features, test_labels, verbose=0)

# Visualizing the accuracy of the linear model.
plt.title('Linear NN Loss Plot')
plot_loss(history)


# In[33]:


# DNN MODEL!!
```

```
# Create the Deep Neural Network model, containing 2 hidden dense layers using relu nonlinearity. (explain relu) (Model 2)
# As both models use the same training procedure, the compile method is included in the build_and_compile_model function below.
from tensorflow.keras.layers import LeakyReLU
def build_and_compile_model(norm):
  model = keras.Sequential([
      norm,
      layers.Dense(256, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(128, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(64, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(32, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(16, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(8, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(4, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(2, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(1)
  ])
# Compiling the model, specifying the loss and optimizer functions.
  model.compile(loss='mse',
                optimizer=tf.keras.optimizers.Adamax(0.001))
  return model

# Compiling the model using the normalized data, and viewing the results
eight_layer_256to2_dnn = build_and_compile_model(normalizer)
eight_layer_256to2_dnn.summary()


# Train the DNN model, recording the time taken for training.
get_ipython().run_line_magic('time', 'history = eight_layer_256to2_dnn.fit(train_features, train_labels, validation_split=0.2, verbose=0, epochs=1000)')

# Saving the results of the DNN model testing.
```

```
test_results['eight_layer_256to2_dnn'] =
eight_layer_256to2_dnn.evaluate(test_features, test_labels, verbose=0)

# Visualizing the validation loss and loss of the DNN model.
plt.title('Eight Layer 256 to 2 Neuron DNN Loss Plot')
print(plot_loss(history))



# In[34]:



# DNN MODEL!!
# Create the Deep Neural Network model, containing 2 hidden dense layers using
relu nonlinearity. (explain relu) (Model 2)
# As both models use the same training procedure, the compile method is included
in the build_and_compile_model function below.
from tensorflow.keras.layers import LeakyReLU
def build_and_compile_model(norm):
  model = keras.Sequential([
      norm,
      layers.Dense(128, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(64, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(32, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(16, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(1)
  ])
# Compiling the model, specifying the loss and optimizer functions.
  model.compile(loss='mse',
                optimizer=tf.keras.optimizers.Adamax(0.001))
  return model

# Compiling the model using the normalized data, and viewing the results
fourlayer_128to16_dnn = build_and_compile_model(normalizer)
fourlayer_128to16_dnn.summary()
```

```
# Record time taken for training, and train the model.
get_ipython().run_line_magic('time', 'history = fourlayer_128to16_dnn.fit(train_features, train_labels, validation_split=0.2,verbose=0, epochs=1000)')

# Saving the results of the DNN model testing.
test_results['fourlayer_128to16_dnn'] = fourlayer_128to16_dnn.evaluate(test_features, test_labels, verbose=0)

# Visualizing the validation loss and loss of the DNN model.
plt.title('Four Layer 128 to 16 Neuron DNN Loss Plot')
print(plot_loss(history))


# In[35]:


# DNN MODEL!!
# Create the Deep Neural Network model, containing 2 hidden dense layers using relu nonlinearity. (explain relu) (Model 2)
# As both models use the same training procedure, the compile method is included in the build_and_compile_model function below.
from tensorflow.keras.layers import LeakyReLU
def build_and_compile_model(norm):
  model = keras.Sequential([
      norm,
      layers.Dense(256, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(128, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(64, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(32, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(1)
  ])
# Compiling the model, specifying the loss and optimizer functions.
  model.compile(loss='mse',
```

```
                    optimizer=tf.keras.optimizers.Adamax(0.001))
    return model

# Compiling the model using the normalized data, and viewing the results
fourlayer_256to32_dnn = build_and_compile_model(normalizer)
fourlayer_256to32_dnn.summary()



# Record time taken for training
get_ipython().run_line_magic('time', '')
# Train the DNN model.
history = fourlayer_256to32_dnn.fit(
    train_features, train_labels,
    validation_split=0.2,
    verbose=0, epochs=1000)

# Saving the results of the DNN model testing.
test_results['fourlayer_256to32_dnn']                                    =
fourlayer_256to32_dnn.evaluate(test_features, test_labels, verbose=0)

# Visualizing the validation loss and loss of the DNN model by making predictions.
plt.title('Four Layer 256 to 32 Neuron DNN Loss Plot')
plot_loss(history)



# In[45]:



# DNN MODEL!!
# Create the Deep Neural Network model, containing 2 hidden dense layers using
relu nonlinearity. (explain relu) (Model 2)
# As both models use the same training procedure, the compile method is included
in the build_and_compile_model function below.
from tensorflow.keras.layers import LeakyReLU
def build_and_compile_model(norm):
```

```
  model = keras.Sequential([
    norm,
    layers.Dense(128, activation=tf.keras.layers.LeakyReLU()),
    layers.Dense(16, activation=tf.keras.layers.LeakyReLU()),
    layers.Dense(1)
  ])
# Compiling the model, specifying the loss and optimizer functions.
  model.compile(loss='mse',
            optimizer=tf.keras.optimizers.Adamax(0.001))
  return model

# Compiling the model using the normalized data, and viewing the results
twolayer_128to16_dnn = build_and_compile_model(normalizer)
twolayer_128to16_dnn.summary()


# Record time taken for training
get_ipython().run_line_magic('time', '')
# Train the DNN model.
history = twolayer_256to128_dnn.fit(
   train_features, train_labels,
   validation_split=0.2,
   verbose=0, epochs=1000)

# Visualizing the validation loss and loss of the DNN model.
plt.title('Two Layer 128 to 16 Neuron DNN Loss Plot')
plot_loss(history)
# Saving the results of the DNN model testing.
test_results['twolayer_128to16_dnn']                                    =
twolayer_256to128_dnn.evaluate(test_features, test_labels, verbose=0)


# In[38]:
```

```
# DNN MODEL!!
# Create the Deep Neural Network model, containing 2 hidden dense layers using relu nonlinearity. (explain relu) (Model 2)
# As both models use the same training procedure, the compile method is included in the build_and_compile_model function below.
from tensorflow.keras.layers import LeakyReLU
def build_and_compile_model(norm):
  model = keras.Sequential([
      norm,
      layers.Dense(256, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(1)
  ])
# Compiling the model, specifying the loss and optimizer functions.
  model.compile(loss='mse',
                optimizer=tf.keras.optimizers.Adamax(0.001))
  return model

# Compiling the model using the normalized data, and viewing the results
onelayer_256_dnn = build_and_compile_model(normalizer)
onelayer_256_dnn.summary()


# Record time taken for training
get_ipython().run_line_magic('time', '')
# Train the DNN model.
history = onelayer_256_dnn.fit(
    train_features, train_labels,
    validation_split=0.2,
    verbose=0, epochs=1000)

# Visualizing the validation loss and loss of the DNN model.
plt.title('One Layer 256 Neuron DNN Loss Plot')
plot_loss(history)

# Saving the results of the DNN model testing.
```

```
test_results['onelayer_256_dnn'] = onelayer_256_dnn.evaluate(test_features, test_labels, verbose=0)


# In[39]:


# DNN MODEL!!
# Create the Deep Neural Network model, containing 2 hidden dense layers using relu nonlinearity. (explain relu) (Model 2)
# As both models use the same training procedure, the compile method is included in the build_and_compile_model function below.
from tensorflow.keras.layers import LeakyReLU
def build_and_compile_model(norm):
  model = keras.Sequential([
      norm,
      layers.Dense(16,activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(8, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(4, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(2, activation=tf.keras.layers.LeakyReLU()),
      layers.Dense(1)
  ])
# Compiling the model, specifying the loss and optimizer functions.
  model.compile(loss='mse',
                optimizer=tf.keras.optimizers.Adamax(0.001))
  return model

# Compiling the model using the normalized data, and viewing the results
fourlayer_16to2_dnn = build_and_compile_model(normalizer)
fourlayer_16to2_dnn.summary()


# Record time taken for training

history = onelayer_256_dnn.fit(
```

```
    train_features, train_labels,
    validation_split=0.2,
    verbose=0, epochs=100)

    # Visualizing the validation loss and loss of the DNN model.
    get_ipython().run_line_magic('time', '')
    plt.title('Four Layer 16 to 2 Neuron DNN Loss Plot')
    plot_loss(history)
    get_ipython().run_line_magic('time', '')
    # Saving the results of the DNN model testing.
    test_results['fourlayer_16to2_dnn']  =  fourlayer_16to2_dnn.evaluate(test_features,
test_labels, verbose=0)


    # In[40]:


    # PERFORMANCE EVALUATION
    pd.DataFrame(test_results, index=['mse [Cp]']).T


    # In[62]:


    from sklearn.metrics import r2_score
    # MAKING PREDICTIONS
    get_ipython().run_line_magic('time',                    'test_predictions                    =
twolayer_256to128_dnn.predict(test_features).flatten()')

    a = plt.axes(aspect='equal')
    plt.title('Two Layer 256 to 128 Neuron DNN Prediction Plot')
    plt.scatter(test_labels, test_predictions)
    plt.xlabel('True Values [Cp]')
    plt.ylabel('Predictions [Cp]')
    lims = [-4,4]
```

```
plt.xlim(lims)
plt.ylim(lims)
_ = plt.plot(lims, lims)

r2_score(test_labels, test_predictions)



# In[49]:



error = test_predictions - test_labels
plt.hist(error, bins=25)
plt.title('Two Layer 256 to 128 Neuron DNN Error Plot')
plt.xlabel('Prediction Error [Cp]')
_ = plt.ylabel('Count')



# In[50]:



r2_score(test_labels, test_predictions)



# In[63]:



# MAKING PREDICTIONS
get_ipython().run_line_magic('time', 'test_predictions = fourlayer_128to16_dnn.predict(test_features).flatten()')

a = plt.axes(aspect='equal')
plt.title('Four Layer 128 to 16 Neuron DNN Prediction Plot')
plt.scatter(test_labels, test_predictions)
plt.xlabel('True Values [Cp]')
plt.ylabel('Predictions [Cp]')
```

```
lims = [-4,4]
plt.xlim(lims)
plt.ylim(lims)
_ = plt.plot(lims, lims)


# In[52]:


error = test_predictions - test_labels
plt.hist(error, bins=25)
plt.title('Four Layer 128 to 16 Neuron DNN Error Plot')
plt.xlabel('Prediction Error [Cp]')
_ = plt.ylabel('Count')


# In[53]:


from sklearn.metrics import r2_score
r2_score(test_labels, test_predictions)


# In[64]:


# MAKING PREDICTIONS
get_ipython().run_line_magic('time', 'test_predictions = onelayer_256_dnn.predict(test_features).flatten()')

a = plt.axes(aspect='equal')
plt.title('One Layer 256 Neuron DNN Prediction Plot')
plt.scatter(test_labels, test_predictions)
plt.xlabel('True Values [Cp]')
plt.ylabel('Predictions [Cp]')
```

```
plt.xlim(lims)
plt.ylim(lims)
_ = plt.plot(lims, lims)


# In[55]:


error = test_predictions - test_labels
plt.hist(error, bins=25)
plt.title('One Layer 256 Neuron DNN Error Plot')
plt.xlabel('Prediction Error [Cp]')
_ = plt.ylabel('Count')


# In[56]:


from sklearn.metrics import r2_score
r2_score(test_labels, test_predictions)


# In[57]:


from sklearn.metrics import r2_score
test_predictions = fourlayer_256to32_dnn.predict(test_features).flatten()

r2_score(test_labels, test_predictions)


# In[58]:


from sklearn.metrics import r2_score
```

```
test_predictions = eight_layer_256to2_dnn.predict(test_features).flatten()

r2_score(test_labels, test_predictions)


# In[59]:


test_predictions = linear_model.predict(test_features).flatten()

r2_score(test_labels, test_predictions)


# In[60]:


test_predictions = fourlayer_16to2_dnn.predict(test_features).flatten()

r2_score(test_labels, test_predictions)


# In[61]:


# Save the DNN model.
dnn_model.save('dnn_model')


# In[ ]:


# Reload the model.
reloaded = tf.keras.models.load_model('dnn_model')

test_results['reloaded'] = reloaded.evaluate(
```

```
    test_features, test_labels, verbose=0)


# In[ ]:


# Verify the model output, ensuring the results are identical.
pd.DataFrame(test_results, index=['Mean squared error [Cp]']).T


# In[ ]:


#tune hyperarams erastuner


# In[ ]:
```

## 12.4 ELU Model

```
#!/usr/bin/env python
# coding: utf-8

# In[1]:


import pandas as pd

# Import the following libraries, for slicing.
from sklearn.model_selection import train_test_split

# To manipulate the data set as a data frame, import pandas.
```

```
import pandas as pd

# For management and plotting histograms, import MatplotLib.
import matplotlib.pyplot as plt

# Import the following libraries, for slicing, cross validating and tuning, respectively.
from sklearn.model_selection import train_test_split, cross_val_score, RandomizedSearchCV

# Import the classification report, accuracy score and confusion matrix functions, to evaluate the model.
from sklearn.metrics import accuracy_score

# Import the pprint function, to print python data structures as inputs.
from pprint import pprint

# To define the arrays for variables, import NumPy.
import numpy as np

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.metrics import r2_score

# Make numpy printouts easier to read.
np.set_printoptions(precision=3, suppress=True)

from tensorflow.keras.layers import ELU


# In[2]:


import tensorflow as tf
```

```
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.layers.experimental import preprocessing

print(tf.__version__)


# In[3]:


#Import the data.
df = pd.read_pickle('Appended data.pickle')


# In[4]:


# Visualize the distibution of the data.
g = sns.pairplot((df), diag_kind='kde')
g.fig.suptitle("Distribution of Airfoil Data", y=1.08)
plt.show()


# In[5]:


# Use the corr function to calculate the correlations within the dataset, and define them for use in a heatmap.
corrdata = df.corr()

# Plot a heatmap to show the correlation of the data within the dataset.
plt.title("Correlations within Airfoil Data")
sns.heatmap(corrdata)
plt.show()
```

```
# In[6]:


# Splitting the data nto training and test sets.
train_dataset = df.sample(frac=0.8, random_state=0)
test_dataset = df.drop(train_dataset.index)


# In[7]:


# Inspect the statistics of the training data.
print (train_dataset.describe().transpose())
# Previewing the training data.
print(train_dataset)
# Visualize the distibution of the data.
sns.pairplot(train_dataset[['Cp', 'x', 'y']], diag_kind='kde')
# Inspect the statistics of the test data.
print (test_dataset.describe().transpose())
# Add titles using /n
print(test_dataset)
# Visualize the distibution of the data.
sns.pairplot(test_dataset[['Cp', 'x', 'y']], diag_kind='kde')


# In[8]:


# Visualize the distibution of the training data.
h = sns.pairplot(train_dataset[['Cp', 'x', 'y']], diag_kind='kde')
h.fig.suptitle("Distribution of Airfoil Training Data", y=1.08)
plt.show()
```

```
# In[9]:


# Visualize the distibution of the data.
i = sns.pairplot(test_dataset[['Cp', 'x', 'y']], diag_kind='kde')
i.fig.suptitle("Distribution of Airfoil Test Data", y=1.08)
plt.show()


# In[10]:


# Splitting the feature set from the label set.
train_features = train_dataset.copy()
test_features = test_dataset.copy()

train_labels = train_features.pop('Cp')
test_labels = test_features.pop('Cp')


# In[11]:


# Normalization of the data. Increases model stability.
# Inspect the ranges of each feature
train_dataset.describe().transpose()[['mean', 'std']]


# In[12]:


# Create the integrated normalization layer.
normalizer = preprocessing.Normalization()
```

```
# In[13]:


# Adapt the newly created layer to the dataset.
normalizer.adapt(np.array(train_features))


# In[14]:


# The adaptation calculates and stores the variance and mean of the dataset.
print(normalizer.mean.numpy())


# In[15]:


# Calling the layer to return the normalized features.
first = np.array(train_features[:1])

with np.printoptions(precision=2, suppress=True):
  print('First example:', first)
  print()
  print('Normalized:', normalizer(first).numpy())


# In[16]:


# Calling a batch of inputs to verify the model is producing the expected data.
#print(linear_model.predict(train_features[:10]))

# Calling the model to initialize its weight matrices and verifying the shape of the
kernel.
```

```
#print(linear_model.layers[1].kernel)


# In[ ]:


#LINEAR MODEL
# Create the linear regression model. (Model 1)
linear_model = tf.keras.Sequential([normalizer,layers.Dense(units=1)])

# Configuration of the training model by specifying the loss and optimizer functions.
linear_model.compile(
    optimizer=tf.optimizers.Adamax(learning_rate=0.001),
    loss='mse')

# Execution of the training of the linear model, Suppressing logging and calculate validation results for 20% of the training data.
get_ipython().run_line_magic('time', 'history = linear_model.fit(train_features, train_labels, epochs=10000, verbose=0, validation_split = 0.2)')

# Visualization of the progress of training the model.
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

def plot_loss(history):
  plt.plot(history.history['loss'], label='loss')
  plt.plot(history.history['val_loss'], label='val_loss')
  plt.ylim([0, 0.5])
  plt.xlabel('Epoch')
  plt.ylabel('Error [Cp]')
  plt.legend()
  plt.grid(True)

# Define the test results dataframe and save the initial model's mean squared error.
test_results = {}
```

```
test_results['linear_model'] = linear_model.evaluate(test_features, test_labels, verbose=0)

# Visualizing the accuracy of the linear model.
plt.title('Linear NN Loss Plot')
plot_loss(history)


# In[19]:


pip install keras


# In[24]:


from tensorflow.keras.metrics import RootMeanSquaredError


# In[52]:


# DNN MODEL!!
# Create the Deep Neural Network model, containing 2 hidden dense layers using relu nonlinearity. (explain relu) (Model 2)
# As both models use the same training procedure, the compile method is included in the build_and_compile_model function below.
from tensorflow.keras.layers import LeakyReLU
def build_and_compile_model(norm):
  model = keras.Sequential([
      norm,
      layers.Dense(256, activation=ELU(alpha=1.0)),
      layers.Dense(128, activation=ELU(alpha=1.0)),
      layers.Dense(64, activation=ELU(alpha=1.0)),
```

```
        layers.Dense(32, activation=ELU(alpha=1.0)),
        layers.Dense(16, activation=ELU(alpha=1.0)),
        layers.Dense(8, activation=ELU(alpha=1.0)),
        layers.Dense(4, activation=ELU(alpha=1.0)),
        layers.Dense(2, activation=ELU(alpha=1.0)),
        layers.Dense(1)
    ])
  # Compiling the model, specifying the loss and optimizer functions.
    model.compile(loss='mse',
                  optimizer=tf.keras.optimizers.Adamax(0.001))
    return model

  # Compiling the model using the normalized data, and viewing the results
  eight_layer_256to2_dnn = build_and_compile_model(normalizer)
  eight_layer_256to2_dnn.summary()

  # Train the DNN model, recording the time taken for training.
  get_ipython().run_line_magic('time', 'history = eight_layer_256to2_dnn.fit(train_features, train_labels, validation_split=0.2, verbose=0, epochs=1000)')

  # Saving the results of the DNN model testing.
  test_results['eight_layer_256to2_dnn'] = eight_layer_256to2_dnn.evaluate(test_features, test_labels, verbose=0)

  # Visualizing the validation loss and loss of the DNN model.
  plt.title('Eight Layer 256 to 2 Neuron DNN Loss Plot')
  print(plot_loss(history))


  # In[53]:


  # DNN MODEL!!
```

```
# Create the Deep Neural Network model, containing 2 hidden dense layers using relu nonlinearity. (explain relu) (Model 2)
# As both models use the same training procedure, the compile method is included in the build_and_compile_model function below.
from tensorflow.keras.layers import LeakyReLU
def build_and_compile_model(norm):
  model = keras.Sequential([
      norm,
      layers.Dense(128, activation=ELU(alpha=1.0)),
      layers.Dense(64, activation=ELU(alpha=1.0)),
      layers.Dense(32, activation=ELU(alpha=1.0)),
      layers.Dense(16, activation=ELU(alpha=1.0)),
      layers.Dense(1)
  ])
# Compiling the model, specifying the loss and optimizer functions.
  model.compile(loss='mse',
                optimizer=tf.keras.optimizers.Adamax(0.001))
  return model

# Compiling the model using the normalized data, and viewing the results
fourlayer_128to16_dnn = build_and_compile_model(normalizer)
fourlayer_128to16_dnn.summary()

# Train the DNN model, recording the time taken for training.
get_ipython().run_line_magic('time', 'history = fourlayer_128to16_dnn.fit(train_features, train_labels, validation_split=0.2, verbose=0, epochs=1000)')

# Saving the results of the DNN model testing.
test_results['fourlayer_128to16_dnn'] = fourlayer_128to16_dnn.evaluate(test_features, test_labels, verbose=0)

# Visualizing the validation loss and loss of the DNN model.
plt.title('Four Layer 128 to 16 Neuron DNN Loss Plot')
print(plot_loss(history))
```

```
# In[54]:


# DNN MODEL!!
# Create the Deep Neural Network model, containing 2 hidden dense layers using relu nonlinearity. (explain relu) (Model 2)
# As both models use the same training procedure, the compile method is included in the build_and_compile_model function below.
from tensorflow.keras.layers import LeakyReLU
def build_and_compile_model(norm):
  model = keras.Sequential([
      norm,
      layers.Dense(256, activation=ELU(alpha=1.0)),
      layers.Dense(128, activation=ELU(alpha=1.0)),
      layers.Dense(64, activation=ELU(alpha=1.0)),
      layers.Dense(32, activation=ELU(alpha=1.0)),
      layers.Dense(1)
  ])
# Compiling the model, specifying the loss and optimizer functions.
  model.compile(loss='mse',
                optimizer=tf.keras.optimizers.Adamax(0.001))
  return model

# Compiling the model using the normalized data, and viewing the results
fourlayer_256to32_dnn = build_and_compile_model(normalizer)
fourlayer_256to32_dnn.summary()

# Train the DNN model, recording the time taken for training.
get_ipython().run_line_magic('time', 'history = fourlayer_256to32_dnn.fit(train_features, train_labels, validation_split=0.2, verbose=0, epochs=1000)')

# Saving the results of the DNN model testing.
```

```
test_results['fourlayer_256to32_dnn'] =
fourlayer_256to32_dnn.evaluate(test_features, test_labels, verbose=0)

# Visualizing the validation loss and loss of the DNN model by making predictions.
plt.title('Four Layer 256 to  Neuron DNN Loss Plot')
plot_loss(history)



# In[72]:



# DNN MODEL!!
# Create the Deep Neural Network model, containing 2 hidden dense layers using
relu nonlinearity. (explain relu) (Model 2)
# As both models use the same training procedure, the compile method is included
in the build_and_compile_model function below.
from tensorflow.keras.layers import ELU
def build_and_compile_model(norm):
  model = keras.Sequential([
      norm,
      layers.Dense(128, activation=ELU(alpha=1.0)),
      layers.Dense(16, activation=ELU(alpha=1.0)),
      layers.Dense(1)])
# Compiling the model, specifying the loss and optimizer functions.
  model.compile(loss='mse',
                optimizer=tf.keras.optimizers.Adamax(0.001))
  return model

# Compiling the model using the normalized data, and viewing the results
twolayer_128to16_dnn = build_and_compile_model(normalizer)
twolayer_128to16_dnn.summary()

# Train the DNN model, recording the time taken for training.
```

```
get_ipython().run_line_magic('time', 'history = twolayer_128to16_dnn.fit(train_features, train_labels, validation_split=0.2, verbose=0, epochs=1000)')

# Visualizing the validation loss and loss of the DNN model.
plt.title('Two Layer 128 to 16 Neuron DNN Loss Plot')
plot_loss(history)
# Saving the results of the DNN model testing.
test_results['twolayer_128to16_dnn'] = twolayer_256to128_dnn.evaluate(test_features, test_labels, verbose=0)


# In[57]:


# DNN MODEL!!
# Create the Deep Neural Network model, containing 2 hidden dense layers using relu nonlinearity. (explain relu) (Model 2)
# As both models use the same training procedure, the compile method is included in the build_and_compile_model function below.
from tensorflow.keras.layers import LeakyReLU
def build_and_compile_model(norm):
  model = keras.Sequential([
      norm,
      layers.Dense(256, activation=ELU(alpha=1.0)),
      layers.Dense(1)
  ])
# Compiling the model, specifying the loss and optimizer functions.
  model.compile(loss='mse',
                optimizer=tf.keras.optimizers.Adamax(0.001))
  return model

# Compiling the model using the normalized data, and viewing the results
onelayer_256_dnn = build_and_compile_model(normalizer)
onelayer_256_dnn.summary()
```

```
# Train the DNN model, recording the time taken for training.
get_ipython().run_line_magic('time', 'history = onelayer_256_dnn.fit(train_features, train_labels, validation_split=0.2, verbose=0, epochs=1000)')

# Visualizing the validation loss and loss of the DNN model.
plt.title('One Layer 256 Neuron DNN Loss Plot')
plot_loss(history)

# Saving the results of the DNN model testing.
test_results['onelayer_256_dnn'] = onelayer_256_dnn.evaluate(test_features, test_labels, verbose=0)


# In[58]:


# DNN MODEL!!
from tensorflow.keras.layers import LeakyReLU
def build_and_compile_model(norm):
  model = keras.Sequential([
      norm,
      layers.Dense(16, activation=ELU(alpha=1.0)),
      layers.Dense(8, activation=ELU(alpha=1.0)),
      layers.Dense(4, activation=ELU(alpha=1.0)),
      layers.Dense(2, activation=ELU(alpha=1.0)),
      layers.Dense(1)
  ])
# Compiling the model, specifying the loss and optimizer functions.
  model.compile(loss='mse',
                optimizer=tf.keras.optimizers.Adamax(0.001))
  return model

# Compiling the model using the normalized data, and viewing the results
fourlayer_16to2_dnn = build_and_compile_model(normalizer)
```

```
fourlayer_16to2_dnn.summary()

# Train the DNN model, recording the time taken for training.
get_ipython().run_line_magic('time', 'history = onelayer_256_dnn.fit(train_features, train_labels, validation_split=0.2, verbose=0, epochs=100)')

# Visualizing the validation loss and loss of the DNN model.
get_ipython().run_line_magic('time', '')
plt.title('Four Layer 16 to 2 Neuron DNN Loss Plot')
plot_loss(history)
get_ipython().run_line_magic('time', '')
# Saving the results of the DNN model testing.
test_results['fourlayer_16to2_dnn'] = fourlayer_16to2_dnn.evaluate(test_features, test_labels, verbose=0)


# In[59]:


# PERFORMANCE EVALUATION
pd.DataFrame(test_results, index=['mse [Cp]']).T


# In[79]:


# MAKING PREDICTIONS
get_ipython().run_line_magic('time', 'test_predictions = twolayer_256to128_dnn.predict(test_features).flatten()')

r2_score(test_labels, test_predictions)


# In[80]:
```

```
# MAKING PREDICTIONS
get_ipython().run_line_magic('time', 'test_predictions = fourlayer_128to16_dnn.predict(test_features).flatten()')

a = plt.axes(aspect='equal')
plt.title('Four Layer 128 to 16 Neuron DNN Prediction Plot')
plt.scatter(test_labels, test_predictions)
plt.xlabel('True Values [Cp]')
plt.ylabel('Predictions [Cp]')
lims = [-4,4]
plt.xlim(lims)
plt.ylim(lims)
_ = plt.plot(lims, lims)

r2_score(test_labels, test_predictions)


# In[81]:


error = test_predictions - test_labels
plt.hist(error, bins=25)
plt.title('Four Layer 128 to 16 Neuron DNN Error Plot')
plt.xlabel('Prediction Error [Cp]')
_ = plt.ylabel('Count')


# In[82]:


# MAKING PREDICTIONS
get_ipython().run_line_magic('time', 'test_predictions = onelayer_256_dnn.predict(test_features).flatten()')
```

```
r2_score(test_labels, test_predictions)


# In[83]:


get_ipython().run_line_magic('time', 'test_predictions = fourlayer_256to32_dnn.predict(test_features).flatten()')

a = plt.axes(aspect='equal')
plt.title('Four Layer 256 to 32 Neuron DNN Prediction Plot')
plt.scatter(test_labels, test_predictions)
plt.xlabel('True Values [Cp]')
plt.ylabel('Predictions [Cp]')
plt.xlim(lims)
plt.ylim(lims)
_ = plt.plot(lims, lims)

r2_score(test_labels, test_predictions)


# In[84]:


error = test_predictions - test_labels
plt.hist(error, bins=25)
plt.title('Four Layer 256 to 32 Neuron DNN Error Plot')
plt.xlabel('Prediction Error [Cp]')
_ = plt.ylabel('Count')


# In[85]:
```

```
get_ipython().run_line_magic('time', 'test_predictions = eight_layer_256to2_dnn.predict(test_features).flatten()')

a = plt.axes(aspect='equal')
plt.title('Eight Layer 256 to 2 Neuron DNN Prediction Plot')
plt.scatter(test_labels, test_predictions)
plt.xlabel('True Values [Cp]')
plt.ylabel('Predictions [Cp]')
plt.xlim(lims)
plt.ylim(lims)
_ = plt.plot(lims, lims)

r2_score(test_labels, test_predictions)


# In[86]:


error = test_predictions - test_labels
plt.hist(error, bins=25)
plt.title('Eight Layer 256 to 2 Neuron DNN Error Plot')
plt.xlabel('Prediction Error [Cp]')
_ = plt.ylabel('Count')


# In[87]:


get_ipython().run_line_magic('time', 'test_predictions = linear_model.predict(test_features).flatten()')

r2_score(test_labels, test_predictions)


# In[88]:
```

```
get_ipython().run_line_magic('time', 'test_predictions = fourlayer_16to2_dnn.predict(test_features).flatten()')

r2_score(test_labels, test_predictions)


# In[70]:


# Save the DNN model.
dnn_model.save('dnn_model')


# In[ ]:


# Reload the model.
reloaded = tf.keras.models.load_model('dnn_model')

test_results['reloaded'] = reloaded.evaluate(
    test_features, test_labels, verbose=0)


# In[ ]:


# Verify the model output, ensuring the results are identical.
pd.DataFrame(test_results, index=['Mean squared error [Cp]']).T


# In[ ]:
```

```
#tune hyperarams erastuner
```

```
# In[ ]:
```

```
# In[ ]:
```